

Sr no.	Topic	Duration (Mins)	Session No (2 Hours)	Session No (4 Hours)
1	Selenium and its Components	4 hrs	1	1
			2	
2	Selenium WebDriver and Locators	2 hrs	3	2
3	Interacting with Web Elements Part-I and Waits in Selenium	4 hrs	4	2
			5	3
4	Interacting with Web Elements Part-II	4 hrs	6	3
			7	4
5	Interacting with Web Elements Part-III	4 hrs	8	4
			9	5
6	Robot Class and use of AutoIt tool to handle window operations	4 hrs	10	5
			11	6
7		6 hrs	12	6

	TestNG Framework		13	7
			14	
8	Automation Frameworks-I	2 hrs	15	8
9	Automation Frameworks-II	2 hrs	16	8
10	Page Object Model	2 hrs	17	9
11	BDD Framework with Cucumber and Selenium Grid	6 hrs	18	9
			19	10
			20	

TOPICS

1. Selenium and its Components

Information

I. What is Automation testing ?

- Automation testing is a software testing technique that involves the use of automated tools and scripts to execute test cases and compare actual outcomes with expected results.
- The primary goal of automation testing is to enhance the efficiency and accuracy of the testing process by reducing manual intervention.
- In automation testing, testers write test scripts using programming languages or test automation tools.
- These scripts simulate user interactions with the application under test, such as clicking buttons, filling forms, and validating outputs.
- Automated tests can be run repeatedly, providing quick feedback on the application's behavior and identifying defects early in the development process.

Automation testing offers several advantages, including:

1. **Faster Testing:** Automated tests can execute much faster than manual tests, enabling more extensive test coverage in a shorter time.

2. Consistency: Automation ensures that tests are consistently executed, reducing the chances of human errors during repetitive tasks.
 3. Reusability: Test scripts can be reused across different test cases, making maintenance more manageable and saving time in the long run.
 4. Scalability: Automation allows running tests on multiple configurations and environments, making it easier to test complex applications.
 5. Regression Testing: Automation is well-suited for regression testing, where tests are repeatedly executed to check if new code changes have introduced any new defects.
- However, it is essential to note that not all testing can or should be automated.
 - Some scenarios, such as exploratory testing, usability testing, and early-stage testing, are better suited for manual testing.
 - Overall, automation testing complements manual testing and is a crucial part of the software testing life cycle, helping teams deliver high-quality software more efficiently.

II. Need for automation testing

- The need for automation testing arises from the challenges and limitations of manual testing in modern software development.

- Automation testing provides several benefits that make it an essential aspect of the testing process.

Here are some key reasons for the need for automation testing:

1. **Speed and Efficiency:** Automation testing significantly reduces the time required to execute test cases compared to manual testing. Automated tests can be run continuously, allowing for quick feedback on code changes and accelerating the software development lifecycle.
2. **Repetitive Test Execution:** In software development, there are many repetitive test scenarios, such as regression testing, where the same test cases need to be executed repeatedly. Automation ensures consistent and reliable execution of these tests, saving valuable time for testers.
3. **Improved Test Coverage:** Automated tests can cover a vast number of test scenarios that may be impractical to test manually. This extensive test coverage helps identify defects and ensures that the application functions correctly across various environments and configurations.
4. **Consistency and Accuracy:** Automation eliminates the possibility of human errors that may occur during manual testing. Automated tests execute precisely as programmed, providing accurate and reliable results.

5. **Cost Savings:** While automation testing may require initial investment in tools and infrastructure, it leads to long-term cost savings. Automated tests can be reused, reducing the need for repeated manual testing and minimizing the risk of defects in production.
6. **Continuous Integration and Continuous Delivery (CI/CD):** Automation testing is a critical enabler of CI/CD pipelines. It allows for automated testing of each code change, ensuring that new features and fixes are thoroughly tested before deployment.
7. **Scalability:** As software applications grow and become more complex, manual testing becomes increasingly challenging and time-consuming. Automation allows for scalable and parallel testing, making it suitable for large and complex projects.
8. **Enhanced Test Reporting and Analysis:** Automation testing tools provide detailed test reports and analytics, allowing testers and stakeholders to analyze test results and identify trends and patterns quickly.
9. **Testing in Different Environments:** Automation testing can simulate different environments, browsers, devices, and operating systems, ensuring compatibility across various platforms.
10. **Agile and DevOps Practices:** Automation testing aligns well with Agile and DevOps methodologies, where frequent and rapid software releases require efficient and continuous testing.

- In conclusion, automation testing is essential for modern software development to meet the demands of faster release cycles, ensure consistent quality, and enhance overall efficiency in the testing process.
- It complements manual testing and enables software development teams to deliver high-quality products to market more rapidly and effectively.

III. When we can automate software application

- Software applications can be automated when certain conditions are met, making automation testing feasible and beneficial.

Here are some common scenarios when automation testing can be implemented:

1. **Stable Requirements:** The software application's requirements should be well-defined and stable. Frequent changes in requirements can lead to high maintenance efforts for automated test scripts.
2. **Repeated Test Scenarios:** When test scenarios need to be executed repeatedly, such as regression testing, automation can save time and effort by automating the repetitive tasks.

3. **Large Test Data:** For applications that require testing with a large dataset, automation can efficiently execute tests and analyze results.
4. **Frequent Regression Testing:** In Agile and DevOps environments, where continuous integration and frequent deployments are common, automation is essential for fast and efficient regression testing.
5. **High-Risk Areas:** Critical and high-risk areas of the application that need rigorous testing can be automated to ensure thorough coverage.
6. **Load and Performance Testing:** Automation tools are commonly used for load and performance testing to simulate multiple user interactions and measure system performance under various scenarios.
7. **Cross-Browser and Cross-Platform Testing:** Automation allows testing across multiple browsers, operating systems, and devices to ensure application compatibility.
8. **Time Constraints:** When there are tight project timelines, automation helps in quick test execution and reduces the overall testing time.
9. **Consistency and Accuracy:** Automation eliminates the risk of human errors and ensures consistency and accuracy in test execution.

10. Parallel Testing: Automation enables parallel test execution, making it suitable for large-scale and complex applications.

- It's important to note that not all aspects of testing can or should be automated.
- There are cases where manual testing is more appropriate, such as exploratory testing, usability testing, and certain user experience evaluations.
- The decision to automate testing should be based on factors like cost-effectiveness, ROI, test coverage, and project requirements.
- Automation testing complements manual testing and should be used judiciously, focusing on repetitive, time-consuming, and critical test scenarios to achieve better test coverage, faster feedback, and improved software quality.

IV. Which test case we can automate

- Test cases that are suitable for automation typically possess certain characteristics that make them suitable candidates.

Here are some types of test cases that are well-suited for automation:

1. Regression Tests: Test cases that need to be executed frequently to ensure that new code changes have not introduced any defects in previously working functionality.

2. Repetitive Test Scenarios: Test cases that require the same steps to be performed multiple times, such as data-driven tests or tests with various input combinations.
3. Load and Performance Testing: Test cases that simulate a large number of users or transactions to assess the performance and scalability of the application.
4. Cross-Browser and Cross-Platform Testing: Test cases that need to be executed across different browsers, devices, or operating systems.
5. Data-Driven Tests: Test cases that require multiple sets of data to be tested against the same steps.
6. Smoke Tests: Basic tests that are executed after each build to quickly assess if the application is stable enough for further testing.
7. Tests on Stable Functionality: Test cases covering stable functionalities that are not likely to change frequently.
8. Integration Testing: Test cases that verify the interactions between different components or modules of the application.

9. APIs and Web Services Testing: Test cases that validate the functionalities and responses of APIs and web services.

10. End-to-End Testing: Test cases that validate the complete flow of the application, including interactions with databases, APIs, and UI.

- It's important to note that not all test cases should be automated.
- Some test scenarios, such as exploratory testing and usability testing, are best performed manually by human testers.
- Additionally, test cases that require complex user interactions, subjective judgment, or visual evaluation may not be suitable for automation.
- When deciding which test cases to automate, consider factors such as test case complexity, test case stability, ROI of automation, and the overall test automation strategy.
- Automation should be used strategically to maximize its benefits and efficiency in the software testing process.

V. Test Automation Types (Unit Testing, API Testing, GUI Testing)

1. Unit Testing:

- a. Unit testing is a type of test automation that focuses on testing individual units or components of software in isolation.
- b. It involves writing automated test cases for small units of code, such as functions, methods, or classes, to ensure that they function correctly.
- c. Unit tests are typically written by developers themselves to verify the behavior of their code and catch any regressions.
- d. Unit testing frameworks, such as JUnit for Java or NUnit for .NET, provide the tools and structure to create and run unit tests efficiently.

2. API Testing:

- a. API (Application Programming Interface) testing is a type of test automation that focuses on testing the communication and integration between different software systems or components.
- b. It involves testing the exposed APIs and verifying that they work as expected, handle various inputs and outputs correctly, and return the appropriate responses.
- c. API testing can be done at different levels, such as unit testing individual API endpoints, integration testing between multiple APIs, or system-level testing that involves testing the entire flow of API requests and responses.
- d. Tools like Postman, RestAssured, or SoapUI are commonly used for API testing.

3. GUI Testing:

- a. GUI (Graphical User Interface) testing is a type of test automation that focuses on testing the functionality, usability, and visual aspects of the user interface of an application.
 - b. It involves automating the interaction with the application's graphical elements, such as buttons, forms, menus, and dialog boxes, to ensure that they behave as expected.
 - c. GUI testing typically involves simulating user actions, such as clicking, typing, or selecting options, and verifying that the application responds correctly.
 - d. Frameworks like Selenium WebDriver, Cypress, or Appium are popular choices for GUI testing.
- Each type of test automation serves a specific purpose and targets different aspects of the software being tested.
 - By employing a combination of unit testing, API testing, and GUI testing, testers can achieve comprehensive test coverage and ensure the quality and reliability of the software.
 - It's important to choose the appropriate type(s) of test automation based on the specific requirements of the project, the nature of the application, and the testing goals.

VI. Test Automation Frameworks

- Test automation frameworks are structures or sets of guidelines that provide a systematic approach to

designing, implementing, and managing automated tests.

- They offer a structured way to organize test code, handle common testing tasks, and promote reusability and maintainability.

1. Keyword-driven Framework:

- a. The keyword-driven framework is a data-driven framework that separates the test automation logic from the test scripts.
- b. It uses a spreadsheet or data file to define keywords or actions that represent specific operations or test steps.
- c. The test scripts are then written using these keywords, allowing non-technical stakeholders or testers to easily create and modify test cases by updating the test data.
- d. This framework promotes code reusability and reduces maintenance efforts.

2. Data-driven Framework:

- a. The data-driven framework separates the test script logic from the test data, allowing testers to create multiple test cases using different sets of data.
- b. Test data is typically stored in external files such as spreadsheets, databases, or XML files.
- c. The test scripts are designed to read the test data dynamically and execute the test steps iteratively for each data set.

- d. This framework enables testers to cover a wide range of scenarios using a single test script, making it scalable and flexible.
3. Behavior-driven Development (BDD) Framework:
- a. BDD frameworks focus on collaboration and communication between technical and non-technical stakeholders.
 - b. They use a human-readable language, such as Gherkin, to describe the expected behavior of the system in the form of scenarios and steps.
 - c. The scenarios are written in a structured format that can be understood by both technical and non-technical team members.
 - d. BDD frameworks like Cucumber or SpecFlow then translate these scenarios into executable automated tests.
 - e. This framework promotes a shared understanding of requirements, encourages collaboration, and facilitates the creation of automated tests that align with business expectations.
- These are just a few examples of test automation frameworks, and there are several others available, such as modular frameworks, hybrid frameworks, and more.
 - The choice of framework depends on factors like project requirements, team skills, application architecture, and testing goals.

- It's important to select a framework that best suits your needs and helps you achieve efficient and effective test automation.

VII. Test Automation Tools

- Test automation tools are software applications that assist in automating the execution of tests, allowing testers to automate repetitive and time-consuming testing tasks.
- These tools provide various features and capabilities to create, execute, and manage automated tests.

1. Selenium:

- a. Selenium is an open-source test automation framework widely used for web application testing.
- b. It provides a suite of tools and libraries that support different programming languages such as Java, Python, C#, etc. Selenium WebDriver is the core component that allows interacting with web elements, simulating user actions, and verifying expected results.
- c. Selenium supports cross-browser testing, parallel test execution, and integrates with various testing frameworks and tools.

2. Appium:

- a. Appium is an open-source test automation tool used for mobile application testing.

- b. It supports automating native, hybrid, and web mobile apps across iOS and Android platforms.
- c. Appium allows testers to write automated tests using programming languages such as Java, JavaScript, Python, etc.
- d. It provides a client-server architecture and uses the WebDriver protocol to interact with mobile devices or emulators.
- e. Appium supports features like gestures, device orientation, multi-touch, and integrates with popular test frameworks.

3. JUnit/TestNG:

- a. JUnit and TestNG are popular test automation frameworks for unit testing in Java.
 - b. They provide annotations, assertions, and other utility classes to create and manage test cases.
 - c. These frameworks support test execution, test grouping, test dependency management, and test reporting.
 - d. They also integrate with build tools like Maven and provide features for parameterized testing, test suites, and parallel test execution.
 - e. JUnit and TestNG are widely used in conjunction with other automation tools or frameworks to build comprehensive test suites.
- There are many other test automation tools available in the market, each with its own set of features and capabilities. Some other notable tools include Cucumber, Robot Framework, SoapUI, and JMeter.

- The selection of a test automation tool depends on factors like project requirements, application type, programming language preference, and team expertise.
- It's important to evaluate and choose a tool that aligns with your testing needs and helps you achieve efficient and effective test automation.

VIII. Test Automation Process explanation

- The test automation process refers to the systematic approach of implementing and executing automated tests.
- It involves several steps that ensure the successful creation, maintenance, and execution of automated tests.

1. Test Planning:

- a. In this phase, the testing objectives, scope, and requirements are defined.
- b. The team determines which test cases are suitable for automation and develops a test automation strategy.

2. Test Tool Selection:

- a. The team selects the appropriate test automation tool or framework based on the project requirements, application type, and team expertise.
- b. The tool should support the desired programming languages, provide necessary

features, and integrate well with other testing tools.

3. Test Script Development:

- a. Test scripts are developed using the chosen test automation tool.
- b. This involves writing code or using a visual interface to create automated test cases.
- c. Test scripts should be modular, maintainable, and reusable to support efficient test automation.

4. Test Data Preparation:

- a. Test data is prepared to be used during automated testing.
- b. This includes creating relevant test data sets, configuring test environment parameters, and ensuring the availability of necessary resources for testing.

5. Test Execution:

- a. Automated test scripts are executed using the selected test automation tool.
- b. The tool interacts with the application or system being tested, simulating user actions and verifying expected results.
- c. Test results are captured and analyzed.

6. Test Result Analysis and Reporting:

- a. The test results are analyzed to identify any failures or issues.

- b. Test reports are generated, providing detailed information about test coverage, passed tests, and failed tests.
- c. This helps in tracking the progress of testing and identifying areas that require further attention.

7. Test Maintenance:

- a. Test scripts are regularly reviewed and updated as the application or system undergoes changes.
- b. Test maintenance involves updating test scripts to reflect changes in the application, fixing any broken or outdated tests, and adding new tests as needed.

8. Continuous Integration and Continuous Testing:

- a. Test automation is integrated into the development and release process through continuous integration and continuous testing practices.
 - b. Automated tests are executed as part of the build and deployment pipeline, ensuring quick feedback on the application's quality.
- Throughout the test automation process, collaboration and communication among team members are essential.
 - Regular feedback and coordination with developers, testers, and other stakeholders help ensure the effectiveness of the automated testing effort.
 - It's important to note that the test automation process may vary depending on the specific project

requirements, tools used, and organizational practices.

- The process should be adaptable and scalable to accommodate the evolving needs of the project and promote efficient and reliable test automation.

IX. Components of Selenium Suite

- The Selenium suite consists of several components that work together to enable automated web testing.
- Each component serves a specific purpose and provides different functionalities.

1. Selenium WebDriver:

- a. Selenium WebDriver is the core component of the Selenium suite.
- b. It provides a programming interface to interact with web browsers.
- c. WebDriver allows you to automate user actions such as clicking buttons, filling forms, and navigating through web pages.
- d. It supports multiple programming languages, including Java, Python, C#, and more.

2. Selenium IDE:

- a. Selenium IDE (Integrated Development Environment) is a browser extension that allows users to record, edit, and playback test scripts.
- b. It provides a simple interface for creating automated tests without the need for programming knowledge.

- c. Selenium IDE is primarily used for quick prototyping and creating basic test cases.

3. Selenium Grid:

- a. Selenium Grid allows you to distribute your tests across multiple machines or browsers for parallel execution.
- b. It enables you to run tests on different operating systems, browsers, and versions simultaneously, which helps to speed up the testing process and improve test coverage.
- c. Selenium Grid is useful for testing in different environments and for running tests on a larger scale.

4. Selenium Remote Control (RC):

- a. Selenium RC is the predecessor to Selenium WebDriver and is now deprecated.
 - b. It provides a way to interact with web browsers using a JavaScript-based automation framework.
 - c. However, Selenium RC is no longer actively maintained and has been replaced by Selenium WebDriver.
- These components work together to provide a comprehensive set of tools and functionalities for automated web testing using Selenium.
 - The choice of which components to use depends on the specific requirements of your testing project, the programming language you prefer, and the scale of your testing needs.

- Selenium WebDriver is the most widely used component and is recommended for most web testing scenarios.

X. Selenium vs. Other Testing Tools

- Selenium is a popular open-source framework for automating web browsers and is widely used for web application testing.
- While there are several other testing tools available in the market, here is a comparison between Selenium and some of the commonly used testing tools:

1. Selenium vs. HP UFT (Unified Functional Testing):

- a. Selenium is an open-source tool, while HP UFT is a commercial tool.
- b. Selenium supports multiple programming languages, whereas UFT uses VBScript for test automation.
- c. Selenium is primarily focused on web application testing, while UFT supports a wider range of technologies and platforms.
- d. Selenium requires more technical expertise to set up and use, while UFT provides a more user-friendly and codeless testing experience.

2. Selenium vs. Appium:

- a. Selenium is used for web application testing, whereas Appium is specifically designed for mobile application testing.

- b. Selenium supports multiple web browsers, while Appium supports both Android and iOS platforms for mobile testing.
- c. Selenium uses WebDriver for browser automation, while Appium uses WebDriver for mobile automation.
- d. Selenium requires a separate setup for each browser, while Appium allows testing across multiple devices and platforms using a single codebase.

3. Selenium vs. JUnit/TestNG:

- a. Selenium is a web testing framework, while JUnit and TestNG are unit testing frameworks for Java.
- b. Selenium integrates with JUnit and TestNG for test execution and reporting purposes.
- c. JUnit and TestNG are primarily used for unit testing individual code components, while Selenium is used for end-to-end functional testing of web applications.

4. Selenium vs. Cucumber:

- a. Selenium is a tool for browser automation and testing, while Cucumber is a BDD (Behavior-Driven Development) framework for writing test scenarios in a human-readable format.
- b. Selenium can be used with Cucumber to define and execute test scenarios using Gherkin syntax.

- c. Cucumber provides a higher-level abstraction for defining test scenarios, making them more readable and understandable for non-technical stakeholders.
- It's important to consider the specific requirements of your project, the nature of the application being tested, and the expertise of your team when choosing between Selenium and other testing tools.
- Each tool has its own strengths and weaknesses, and the selection should be based on factors such as the type of application, testing objectives, available resources, and desired level of automation.

XI. What is Selenium IDE?

- Selenium IDE (Integrated Development Environment) is a record and playback tool for creating automated test scripts in Selenium.
- It is a browser extension available for Google Chrome and Firefox that allows testers and developers to easily create, edit, and execute automated tests without the need for any programming knowledge.
- Selenium IDE provides a user-friendly interface where testers can record their interactions with a web application, such as clicking buttons, filling forms, navigating pages, and verifying elements.
- The tool captures these actions and generates corresponding Selenium commands, known as Selenese, which form the basis of the test script.

Key features of Selenium IDE include:

1. Record and Playback:

- a. Selenium IDE allows users to record their interactions with a web application and play them back as automated test scripts.

2. Test Script Editing:

- a. Users can modify and enhance recorded test scripts using the built-in editor.
- b. They can add assertions, conditional statements, loops, and other commands to make the test scripts more robust.

3. Test Execution:

- a. Selenium IDE provides a simple interface to execute test scripts directly within the tool.
- b. Users can run tests on the same browser or different browsers supported by Selenium.

4. Test Debugging:

- a. The tool offers debugging capabilities to help identify and resolve issues in test scripts.
- b. Users can set breakpoints, step through the script, and inspect variables and values during test execution.

5. Export Options:

- a. Selenium IDE allows users to export test scripts in various programming languages, such as Java, C#, Python, and Ruby, which can then be executed using Selenium WebDriver for more advanced test automation.

- While Selenium IDE is a useful tool for quick test script creation and simple test scenarios, it has limitations compared to using Selenium WebDriver with programming languages.
- Selenium IDE is primarily suited for beginners, manual testers, or for creating quick prototypes and proof of concepts.
- It's important to note that Selenium IDE is not supported in all browsers and may have compatibility issues with certain web applications.
- Additionally, its features and capabilities may be limited compared to using Selenium WebDriver directly.
- Therefore, for more complex test automation scenarios and greater flexibility, it is recommended to use Selenium WebDriver with programming languages like Java, C#, Python, or Ruby.

XII. Selenium IDE Interface

- The Selenium IDE (Integrated Development Environment) interface provides a user-friendly environment for creating and executing automated test scripts.
- It offers various panels and features that enable testers to record, edit, and execute tests without the need for programming knowledge.

1. Test Case Pane:

- a. This pane displays the test cases created within the Selenium IDE.
- b. Each test case consists of a sequence of steps representing user actions and assertions.

2. Test Step Pane:

- a. This pane shows the individual steps within a selected test case.
- b. It displays the commands that represent user actions like clicks, typing, navigating, etc.

3. Target/Value Fields:

- a. These fields are used to specify the target element on the web page and the value to be entered or verified.
- b. Testers can use various element selection methods like XPath, CSS selectors, etc., to identify the target element.

4. Command Toolbar:

- a. The command toolbar provides a list of commands that testers can add to the test steps.
- b. It includes commands for common actions like clicking, typing, selecting options, assertions, etc.

5. Log Pane:

- a. The log pane displays the execution log of the test script, including information about the executed commands, success or failure status, and any error messages encountered during test execution.

6. Test Playback Controls:

- a. The playback controls allow testers to execute, pause, resume, or stop the execution of the test script.
- b. Testers can also set breakpoints and step through the test script for debugging purposes.

7. Side Panes:

- a. The side panes in Selenium IDE provide additional functionality and information.
- b. This includes the Test Suites pane for organizing and executing multiple test cases, the Variables pane for managing variables used in the test script, and the Plugins pane for managing installed plugins.

8. Options and Preferences:

- a. Selenium IDE offers options and preferences settings that allow testers to configure various aspects of test script recording, playback, and other behaviors.

9. Export and Save Options:

- a. Testers can save their test scripts in different formats, such as HTML, JSON, or a Selenium WebDriver script in a programming language.
- b. They can also export the test script to various programming languages for further customization and execution.

- The Selenium IDE interface is designed to be intuitive and user-friendly, providing a visual and interactive way to create and execute automated tests.
- It enables testers to quickly build and validate test cases without the need for complex programming knowledge or a separate development environment.

XIII. Selenese in Selenium IDE

- Selenese is the scripting language used in Selenium IDE to create test scripts.
- It consists of a set of commands that represent user actions and assertions.
- Testers can use Selenese commands to interact with web elements, perform actions like clicking, typing, selecting options, and verify expected outcomes.

Key features of Selenese in Selenium IDE:

1. Command Structure:

- a. Selenese commands follow a specific structure. Each command consists of a command name, followed by one or more arguments.
- b. Arguments can be used to specify target elements, values, or other parameters required by the command.

2. User Actions:

- a. Selenese commands allow testers to simulate user actions on web pages.

- b. For example, the "click" command is used to simulate a click on an element, the "type" command is used to enter text into input fields, and the "select" command is used to choose an option from a dropdown menu.

3. Assertions:

- a. Selenese commands also support assertions, which are used to verify expected outcomes.
- b. Assertions allow testers to validate that certain conditions are met during the test execution.
- c. For example, the "assertText" command can be used to check if a specific text is present on a web page.

4. Targeting Elements:

- a. Selenese commands use various methods to target elements on a web page.
- b. Testers can specify elements using attributes like id, name, CSS selectors, XPath, and more.
- c. This allows precise identification of elements for interaction or verification purposes.

5. Verification Points:

- a. Selenese commands provide verification points that help validate the expected behavior of a web application.
- b. Testers can compare actual values with expected values using commands like "verifyText" or "verifyTitle".

- Selenese provides a simple and readable syntax for creating test scripts in Selenium IDE.
- It allows testers to quickly create automated tests without the need for complex programming knowledge.
- By leveraging Selenese commands, testers can efficiently interact with web elements, perform actions, and validate expected outcomes in their web applications.

2. Selenium WebDriver and Locators

Information

I. Introduction to Selenium WebDriver

- Selenium WebDriver is a powerful open-source automation tool widely used for testing web applications.
- It provides a programming interface for interacting with web elements, performing actions, and validating the behavior of web applications.
- Selenium WebDriver supports various programming languages such as Java, Python, C#, Ruby, and more, making it highly versatile.

Key points to understand about Selenium WebDriver:

1. Object-Oriented Approach:

- a. Selenium WebDriver follows an object-oriented approach, allowing testers to write automation scripts using programming languages.
- b. It provides a rich set of methods and classes to interact with web elements, handle browser operations, and perform various actions.

2. Cross-Browser Compatibility:

- a. Selenium WebDriver supports multiple browsers, including Chrome, Firefox, Safari, Edge, and more.
- b. This allows testers to run their tests on different browsers to ensure the compatibility and consistent behavior of web applications across platforms.

3. Locators and Web Elements:

- a. Selenium WebDriver offers a variety of locators, such as ID, name, CSS selector, XPath, and more, to identify web elements on a web page.
- b. Testers can use these locators to interact with buttons, input fields, dropdowns, checkboxes, and other elements.

4. Actions and Interactions:

- a. WebDriver provides methods to perform actions like clicking, typing, selecting options, navigating through pages, handling alerts, and more.
- b. Testers can simulate user interactions with web elements to simulate real-user scenarios.

5. Synchronization:

- a. WebDriver supports explicit and implicit waits to handle synchronization issues.
- b. Explicit waits allow testers to pause the execution until certain conditions are met, ensuring that the web page is fully loaded or an element is visible before proceeding.
- c. Implicit waits set a default waiting time for WebDriver to wait for elements to appear on the page.

6. Multiple Window and Frames:

- a. Selenium WebDriver supports handling multiple browser windows and frames within a web application.
- b. Testers can switch between windows and frames to interact with specific elements or perform actions in different contexts.

7. Test Reporting and Integration:

- a. WebDriver integrates well with various testing frameworks, build tools, and test management systems.
- b. It provides features for generating test reports, capturing screenshots, handling test data, and integrating with Continuous Integration (CI) systems for seamless test execution.

8. Community Support:

- a. Selenium WebDriver has a vast and active community of users and contributors.

- b. It provides extensive documentation, tutorials, forums, and online resources to support testers in learning and using the tool effectively.
 - c. Community support ensures timely updates, bug fixes, and the availability of plugins and extensions to enhance test automation capabilities.
- Selenium WebDriver empowers testers to build robust and scalable automation frameworks for web application testing.
- Its versatility, extensive browser support, and rich programming capabilities make it a popular choice among testers and developers for implementing reliable and efficient test automation solutions.

1. Selenium 3:

- a. Selenium 3 was a major update that focused on stability, security, and future compatibility.
- b. It included enhancements to browser compatibility, better integration with modern browsers, and improved support for mobile testing.
- c. Selenium 3 also introduced the concept of "GeckoDriver" for Firefox browser automation and "ChromeDriver" for Chrome browser automation.

2. Selenium 4:

- a. Selenium 4 is the latest major version of Selenium WebDriver, which brings several new features and enhancements.
 - b. It includes native support for the W3C WebDriver standard, improved Selenium Grid capabilities, relative locators for easier element identification, and the addition of the "DevTools" API for advanced browser automation.
 - c. Selenium 4 aims to provide a more user-friendly and robust automation experience.
- The evolution of Selenium WebDriver has made significant contributions to the field of web application testing automation.
 - It has evolved from a basic JavaScript injection approach to a powerful and versatile tool that supports multiple programming languages, browser compatibility, parallel execution, and advanced features.
 - Selenium WebDriver continues to be a popular choice for automating web applications and has greatly simplified the process of building reliable and efficient test automation solutions.

VI. Selenium WebDriver Architecture

- Selenium WebDriver 4's architecture brings important changes from its predecessor, mainly the full adoption of the W3C WebDriver specification.
- This is a standard that was developed to allow better compatibility and consistency across different web browsers.

Here's an overview of Selenium WebDriver 4's architecture:

1. **Test Script:** This is the script created by the tester in any language supported by Selenium (Java, C#, Python, Ruby, etc.). This is where Selenium commands are written to perform actions on a webpage.
 1. **Selenium WebDriver API:** This is the programming interface used by the tester to connect with the browser. It contains various commands provided by Selenium to perform different actions on the web elements.
 2. **W3C WebDriver Protocol:** This is the standard protocol that replaced the JSON Wire Protocol in Selenium 4. It allows communication between the WebDriver and the browser driver. The Selenium commands are encoded in HTTP requests and are sent to the browser driver.
 3. **Browser Drivers:** Each browser (Chrome, Firefox, Internet Explorer, etc.) has its driver, which acts as a bridge between the WebDriver and the browsers. The driver receives the Selenium commands sent via the W3C WebDriver Protocol. It then interprets and executes these commands on the respective browser and sends the HTTP response back to the WebDriver.
 4. **Browsers:** This is where the actions commanded by the test scripts are performed.
- Here's the interaction between these components:
 - You write and run a test script using one of the Selenium-supported languages.
 - Selenium WebDriver API receives this script.

- WebDriver API communicates with the Browser Driver using the W3C WebDriver Protocol. The commands in the test script are encoded in HTTP requests and sent to the Browser Driver.
- The Browser Driver uses this information to communicate with the actual browser.
- The commands are executed on the browser and the results are returned back along the same path.
- The Browser Driver sends the execution status back to the WebDriver.
- WebDriver finally sends the status back to your test script.
- By adopting the W3C WebDriver standard, Selenium WebDriver 4 allows for better browser compatibility, which means tests should behave more consistently across different web browsers.
- Additionally, WebDriver 4 has removed the need for encoding and decoding of API requests and responses that was required in the JSON Wire Protocol, simplifying the communication process.

I. Selenium Commands in WebDriver-Browser navigation commands

- Selenium WebDriver provides several commands for browser navigation.
- These commands allow you to control the browser's navigation actions, such as opening URLs, navigating back and forward, refreshing the page, and managing browser windows.

1. `get(url)`: Opens the specified URL in the current browser window.

Example: `driver.get("https://www.example.com")`

2. `navigate().to(url)`: Navigates to the specified URL in the current browser window. This command is equivalent to `get(url)`.

Example:

`driver.navigate().to("https://www.example.com")`

3. `navigate().back()`: Navigates back to the previous page in the browser's history.

Example: `driver.navigate().back()`

4. `navigate().forward()`: Navigates forward to the next page in the browser's history.

Example: `driver.navigate().forward()`

5. `navigate().refresh()`: Refreshes the current page.

Example: `driver.navigate().refresh()`

6. `quit()`: Closes the browser and ends the WebDriver session.

Example: `driver.quit()`

- These commands allow you to perform common browser navigation tasks in Selenium WebDriver.
- By using these commands, you can control the browser's behavior and navigate between different pages or perform actions on the current page.

V. Introduction to Web elements

- In the context of web automation testing using Selenium WebDriver, web elements refer to the different components or objects present on a web page that can be interacted with.
- These elements can include buttons, text fields, checkboxes, dropdowns, links, images, and more.
- The ability to identify and interact with web elements is crucial for automating test scenarios.
- Each web element is represented by an HTML tag and can have various attributes such as id, class, name, value, or text content.
- Selenium WebDriver provides methods and techniques to locate and manipulate these web elements in order to perform actions like clicking, entering text, selecting options, or validating their properties.
- The identification and interaction with web elements are typically done using locators.
- Locators are strategies or mechanisms used to uniquely identify a web element on a web page.

Some commonly used locators in Selenium WebDriver are:

1. ID: The unique identifier assigned to an element.
2. Class Name: The CSS class name of an element.
3. Name: The value of the "name" attribute of an element.
4. Tag Name: The HTML tag name of an element.
5. Link Text: The text content of a link element.
6. Partial Link Text: A partial match of the link text.

7. CSS Selector: A CSS-based selector to locate elements.
8. XPath: A powerful language for traversing the XML structure of a web page.

- By using these locators, testers can find web elements on a page and perform various actions on them, such as clicking, sending keys, retrieving text, or verifying their properties.
- Selenium WebDriver provides methods like `findElement(By locator)` and `findElements(By locator)` to locate and interact with web elements based on the chosen locator strategy.
- Overall, web elements are the building blocks of web pages, and understanding how to identify and interact with them is essential for automating test scenarios effectively using Selenium WebDriver.

II. What are CSS Selectors?

- CSS Selectors are patterns used to select and target specific HTML elements on a web page.
- They allow you to apply styles or perform actions on elements based on their attributes, classes, IDs, or their relationship with other elements.
- CSS Selectors provide a powerful and flexible way to select elements and are widely used in web development and automation testing.

1. Element Selector: Selects elements based on their tag name.

Example: `p` selects all `<p>` elements.

2. Class Selector: Selects elements based on their class attribute.

Example: `.my-class` selects all elements with the class "my-class".

3. ID Selector: Selects an element based on its ID attribute.

Example: `#my-id` selects the element with the ID "my-id".

4. Attribute Selector: Selects elements based on their attributes.

Example: `[type='submit']` selects all elements with the attribute "type" equal to "submit".

- CSS Selectors provide a concise and powerful way to target elements in HTML documents.
- They are widely supported by browsers and can be used in various contexts, including styling web pages and locating elements in automated tests using tools like Selenium WebDriver.

VI. Introduction to XPath

- XPath is a query language used to navigate and locate elements within an XML document or an HTML page.
- In the context of web automation testing with Selenium WebDriver, XPath is commonly used as a locator strategy to identify web elements on a web page.
- XPath allows you to traverse the XML or HTML structure of a page and select nodes based on their

attributes, tag names, text content, or relative positions.

- It provides a powerful way to specify the location of elements, even if they don't have unique identifiers.
- XPath expressions are written using a syntax that resembles a path-like structure.

VII. Types of XPath

- There are two types of XPath expressions:

1. Absolute XPath:

- a. Starts with the root node of the document (typically html or /) and specifies the complete path to the desired element.
- b. It uses the complete hierarchy of elements from the root node to the target element.
- c. Example: `/html/body/div[1]/form/input[3]`

2. Relative XPath:

- a. Starts from a specific node and traverses the structure to locate the desired element.
- b. It does not rely on the complete hierarchy and is more flexible as it adapts to changes in the structure of the page.
- c. Relative XPath expressions typically use attributes, relationships, or other characteristics of elements to locate them.
- d. Example: `//input[@id='username']`

- In addition to these types, XPath expressions can also use various axes and functions to filter and refine the selection of elements.

Some commonly used XPath expressions include:

- Selecting elements by tag name: `//div`, `//input`, etc.
- Selecting elements by attribute values:
`//input[@name='email'],`
`//a[@href='https://www.example.com']`
- Selecting elements with specific text content:
`//h1[text()='Welcome'],`
`//span[contains(text(),'Search')]`
- Selecting elements based on their position: `(//a)[3]`,
`//div[@class='container']/div[2]`
- It's important to note that the choice of XPath expression depends on the specific element and its context on the page.
- Relative XPath expressions are generally preferred as they provide more flexibility and are less likely to break with changes to the page structure.
- However, in some cases, absolute XPath may be necessary when the element's position in the hierarchy is crucial.
- When using XPath in Selenium WebDriver, you can use the `By.xpath(xpathExpression)` method to locate elements based on XPath expressions.

VIII. XPath Functions and Customisations

- XPath provides a variety of functions and customizations that can be used to enhance the selection and manipulation of elements.

Some commonly used XPath functions include:

1. `text()`: Selects the text content of an element.

Example: `//div[text()='Hello, World!']`

2. `contains()`: Checks if an attribute value contains a specified substring.

Example: `//input[contains(@class, 'username')]`

3. `starts-with()`: Checks if an attribute value starts with a specified substring.

Example: `//a[starts-with(@href, 'https://example.com')]`

4. `normalize-space()`: Removes leading and trailing whitespace and collapses multiple spaces into a single space.

Example: `normalize-space(//p)`

- These are just a few examples of XPath functions that can be used to refine element selection.
- XPath also supports mathematical functions, comparison operators, logical operators, and more.
- In addition to functions, XPath allows for customizations such as using variables, defining predicates, and combining multiple XPath expressions with logical operators (AND, OR, NOT).
- These features provide flexibility and control when constructing complex XPath expressions.

- When using XPath in Selenium WebDriver, you can incorporate these functions and customizations within your XPath expressions to target specific elements based on various conditions and criteria.

3. Interacting with Web Elements Part-I and Waits in Selenium

Information

I. Selenium Commands in WebDriver-Get commands

- In Selenium WebDriver, the "get" commands are used to retrieve information or properties of web elements on a page.
- These commands allow you to extract data from various elements such as text boxes, buttons, links, and more.

1. `getText()`: Retrieves the visible text of an element.

Example: `String text = element.getText();`

2. `getAttribute(attributeName)`: Retrieves the value of the specified attribute of an element.

Example: `String value = element.getAttribute("value");`

3. `getCssValue(propertyName)`: Retrieves the value of the specified CSS property of an element.

Example: `String color = element.getCssValue("color");`

4. `isEnabled()`: Checks if an element is enabled or disabled.

Example: `boolean isEnabled = element.isEnabled();`

5. `isSelected()`: Checks if a checkbox or radio button is selected.

Example: `boolean isSelected = element.isSelected();`

6. `getLocation()`: Retrieves the location of an element on the page.

Example: `Point location = element.getLocation();`

7. `getSize()`: Retrieves the size of an element.

Example: `Dimension size = element.getSize();`

- These "get" commands allow you to retrieve information from web elements, such as their text content, attribute values, CSS properties, and more.
- They provide valuable data that can be used for verification, validation, or further processing in your test scripts.

III. Interacting with Web Elements

- In Selenium WebDriver, interacting with web elements refers to performing actions such as clicking, typing, selecting options, submitting forms, and more.
- These actions simulate user interactions with the web application being tested.

1. `click()`: Performs a click action on the web element.
Example: `element.click()`;

2. `sendKeys(keysToSend)`: Simulates typing or entering text into an input field.
Example: `element.sendKeys("Hello, World!")`;

3. `clear()`: Clears the text entered in an input field.
Example: `element.clear()`;

4. `submit()`: Submits a form element.
Example: `element.submit()`;

5. `isSelected()`: Checks if a checkbox or radio button is selected.
Example: `boolean isSelected = element.isSelected()`;

6. `isEnabled()`: Checks if an element is enabled or disabled.
Example: `boolean isEnabled = element.isEnabled()`;

- These methods allow you to interact with web elements and perform various actions that mimic user interactions.
- They are essential for testing the functionality and behavior of web applications.

IV. Performing Actions on Web Elements-text box

- To perform actions on a text box in Selenium WebDriver, you can use the following methods:

1. sendKeys(keysToSend):

- a. This method is used to simulate typing or entering text into a text box.
- b. You can pass the text to be entered as a parameter to the method.

Example: WebElement textBox =
driver.findElement(By.id("textbox"));
textBox.sendKeys("Hello, World!");

2. clear(): This method is used to clear the text entered in a text box.

Example: WebElement textBox =
driver.findElement(By.id("textbox")); textBox.clear();

3. getAttribute(attributeName):

- a. This method is used to retrieve the value of a specific attribute of the text box element.
- b. For example, you can retrieve the "value" attribute to get the text entered in the text box.

Example: WebElement textBox =
driver.findElement(By.id("textbox")); String enteredText =
textBox.getAttribute("value"); System.out.println("Entered
text: " + enteredText);

4. isEnabled(): This method is used to check if the text box is enabled or disabled.

Example: WebElement textBox =
driver.findElement(By.id("textbox")); boolean isEnabled =
textBox.isEnabled(); System.out.println("Is text box
enabled? " + isEnabled);

5. `isDisplayed()`: This method is used to check if the text box is displayed on the web page.

Example: `WebElement textBox = driver.findElement(By.id("textbox")); boolean isDisplayed = textBox.isDisplayed(); System.out.println("Is text box displayed? " + isDisplayed);`

- These methods allow you to interact with text boxes on a web page, enter text, clear the existing text, retrieve entered text or attribute values, and check the enabled or displayed status of the text box.

V. Performing Actions on Web Elements-links

To perform actions on links in Selenium WebDriver, you can use the following methods:

1. `click()`: This method is used to click on a link. It simulates the user clicking on the link.

Example: `WebElement link = driver.findElement(By.linkText("Home")); link.click();`

2. `getText()`: This method is used to retrieve the text of a link. It returns the visible text displayed on the link.

Example: `WebElement link = driver.findElement(By.linkText("About Us")); String linkText = link.getText(); System.out.println("Link text: " + linkText);`

3. `getAttribute(attributeName)`: This method is used to retrieve the value of a specific attribute of the link

element. You can retrieve attributes like "href" to get the link URL.

```
Example: WebElement link =  
driver.findElement(By.linkText("Contact")); String linkUrl  
= link.getAttribute("href"); System.out.println("Link URL:  
" + linkUrl);
```

4. isEnabled(): This method is used to check if the link is enabled or disabled.

```
Example: WebElement link =  
driver.findElement(By.linkText("Products")); boolean  
isEnabled = link.isEnabled(); System.out.println("Is link  
enabled? " + isEnabled);
```

5. isDisplayed(): This method is used to check if the link is displayed on the web page.

```
Example: WebElement link =  
driver.findElement(By.linkText("FAQs")); boolean  
isDisplayed = link.isDisplayed(); System.out.println("Is  
link displayed? " + isDisplayed);
```

- These methods allow you to interact with links on a web page, click on them, retrieve link text or attribute values, and check the enabled or displayed status of the link.

VI. Performing Actions on Web Elements-button

To perform actions on buttons in Selenium WebDriver, you can use the following methods:

1. `click()`: This method is used to click on a button. It simulates the user clicking on the button.

Example: `WebElement button =`

```
driver.findElement(By.id("submitBtn")); button.click();
```

2. `getText()`: This method is used to retrieve the text displayed on the button.

Example: `WebElement button =`

```
driver.findElement(By.className("loginBtn")); String  
buttonText = button.getText(); System.out.println("Button  
text: " + buttonText);
```

3. `isEnabled()`: This method is used to check if the button is enabled or disabled.

Example: `WebElement button =`

```
driver.findElement(By.cssSelector("input[type='submit']")  
); boolean isEnabled = button.isEnabled();  
System.out.println("Is button enabled? " + isEnabled);
```

4. `isDisplayed()`: This method is used to check if the button is displayed on the web page.

Example: `WebElement button =`

```
driver.findElement(By.xpath("//button[@id='cancelBtn']")  
); boolean isDisplayed = button.isDisplayed();  
System.out.println("Is button displayed? " + isDisplayed);
```

5. `submit()`: This method is used to submit a form by clicking on the button. It is typically used with `<input>` elements of type "submit".

Example: `WebElement submitButton =`

```
driver.findElement(By.tagName("input"));  
submitButton.submit();
```

- These methods allow you to interact with buttons on a web page, click on them, retrieve button text, check the enabled or displayed status, and submit forms using buttons.

VII. Performing Actions on Web Elements-Radio Button

To perform actions on radio buttons in Selenium WebDriver, you can use the following methods:

1. `click()`: This method is used to click on a radio button. It simulates the user clicking on the radio button.

Example: `WebElement radioButton = driver.findElement(By.id("maleRadio")); radioButton.click();`

2. `isSelected()`: This method is used to check if a radio button is selected or not.

Example: `WebElement radioButton = driver.findElement(By.xpath("//input[@name='gender' and @value='female']")); boolean isSelected = radioButton.isSelected(); System.out.println("Is radio button selected? " + isSelected);`

3. `isEnabled()`: This method is used to check if a radio button is enabled or disabled.

Example: `WebElement radioButton = driver.findElement(By.cssSelector("input[type='radio']")); boolean isEnabled = radioButton.isEnabled(); System.out.println("Is radio button enabled? " + isEnabled);`

4. `isDisplayed()`: This method is used to check if a radio button is displayed on the web page.

```
Example: WebElement radioButton =  
driver.findElement(By.className("radioButton"));  
boolean isDisplayed = radioButton.isDisplayed();  
System.out.println("Is radio button displayed? " +  
isDisplayed);
```

- These methods allow you to interact with radio buttons on a web page, click on them, check their selected status, and check if they are enabled or displayed.

VIII. Performing Actions on Web Elements-Checkboxes

To perform actions on checkboxes in Selenium WebDriver, you can use the following methods:

1. `click()`: This method is used to click on a checkbox. It simulates the user clicking on the checkbox.

```
Example: WebElement checkbox =  
driver.findElement(By.id("agreeCheckbox"));  
checkbox.click();
```

2. `isSelected()`: This method is used to check if a checkbox is selected or not.

```
Example: WebElement checkbox =  
driver.findElement(By.xpath("//input[@id='newsletterChe  
ckbox']")); boolean isSelected = checkbox.isSelected();  
System.out.println("Is checkbox selected? " + isSelected);
```

3. isEnabled(): This method is used to check if a checkbox is enabled or disabled.

Example: WebElement checkbox =
driver.findElement(By.cssSelector("input[type='checkbox']")); boolean isEnabled = checkbox.isEnabled();
System.out.println("Is checkbox enabled? " + isEnabled);

4. isDisplayed(): This method is used to check if a checkbox is displayed on the web page.

Example: WebElement checkbox =
driver.findElement(By.className("checkbox")); boolean isDisplayed = checkbox.isDisplayed();
System.out.println("Is checkbox displayed? " + isDisplayed);

- These methods allow you to interact with checkboxes on a web page, click on them, check their selected status, and check if they are enabled or displayed.

IX. Performing Actions on Web Elements-Dynamic Web Tables

- Performing actions on dynamic web tables in Selenium WebDriver requires a combination of locating the table elements and using various methods to interact with the table data.

Here are some commonly used actions on dynamic web tables:

1. Locating the Table:

- a. Use the appropriate locator strategy (such as XPath, CSS selector, or ID) to locate the table element on the web page.

2. Getting Table Rows:

- a. Once the table element is located, use the `findElements()` method to get all the rows in the table.
- b. This will return a list of `WebElement` objects representing each row.

3. Getting Table Cells:

- a. Iterate through the rows and use the `findElements()` method on each row to get all the cells within that row.
- b. This will return a list of `WebElement` objects representing each cell.

4. Accessing Cell Data:

- a. To access the data within a specific cell, you can use the `getText()` method on the respective `WebElement` representing the cell.
- b. This will retrieve the text content of the cell.

5. Performing Actions:

- a. Based on your testing scenario, you can perform various actions such as reading data, modifying data, or verifying data in the table cells.
- b. You can use conditionals, loops, and other programming constructs to perform these actions dynamically.

Here's an example of iterating through a dynamic web table and printing the content of each cell:

```
WebElement table = driver.findElement(By.id("tableId"));
List<WebElement> rows = table.findElements(By.tagName("tr"));

for (WebElement row : rows) {
    List<WebElement> cells = row.findElements(By.tagName("td"));
    for (WebElement cell : cells) {
        String cellText = cell.getText();
        System.out.println("Cell Text: " + cellText);
    }
}
```

- This code locates the table element, iterates through each row, and then iterates through each cell within the row to retrieve and print the cell text.
- You can modify this example based on your specific requirements to perform different actions on the dynamic web table.

X. Types of Waits in Selenium

- In Selenium, waits are used to ensure that the WebDriver waits for a certain condition to be met before proceeding with the next step.
- This is especially useful when dealing with web applications that have dynamic content or elements that may take some time to load.

There are three types of waits in Selenium:

1. Implicit Waits:

- a. Implicit waits are set at the WebDriver level and apply to all elements.
- b. With an implicit wait, WebDriver will wait for a specified amount of time before throwing an exception if the element is not found or interactable.
- c. This wait is applied automatically to all `findElement` and `findElements` methods.

```
driver.manage().timeouts().implicitlyWait(10, TimeUnit.SECONDS);
```

2. Explicit Waits:

- a. Explicit waits are used to wait for a specific condition to occur before proceeding to the next step.
- b. With explicit waits, you can define a specific condition and the maximum amount of time to wait for that condition to be met.
- c. This provides more control and flexibility compared to implicit waits.

```
WebDriverWait wait = new WebDriverWait(driver, 10);  
WebElement element =  
wait.until(ExpectedConditions.visibilityOfElementLocated(By.id("elementId")));
```

3. Fluent Waits:

- a. Fluent waits are an advanced form of explicit waits that allow you to define a custom polling interval and exceptions to ignore during the wait.
- b. Fluent waits provide a fluent API that allows you to define the conditions and duration for waiting.

```
Wait<WebDriver> wait = new FluentWait<>(driver)
    .withTimeout(Duration.ofSeconds(10))
    .pollingEvery(Duration.ofMillis(500))
    .ignoring(NoSuchElementException.class);
```

```
WebElement element = wait.until(driver ->
    driver.findElement(By.id("elementId")));
```

- By using these different types of waits, you can handle different synchronization scenarios in your Selenium tests, ensuring that your tests wait for the desired conditions to be met before proceeding with the next steps.

4. Interacting with Web Elements Part-II

Information

I. Select Class in Selenium WebDriver

- In Selenium WebDriver, the Select class is used for handling dropdown (select) elements in web applications.

- It provides methods to interact with dropdowns, select options, and retrieve selected values.

Here's an explanation of the Select class and its commonly used methods:

1. Creating a Select object:

```
WebElement dropdownElement =  
driver.findElement(By.id("dropdownId"));  
Select dropdown = new Select(dropdownElement);
```

2. Selecting options:

a. Select by index:

```
dropdown.selectByIndex(index);
```

```
dropdown.selectByVisibleText("Option Text");
```

c. Select by value attribute:

```
dropdown.selectByValue("optionValue");
```

3. Deselecting options (applicable for multi-select dropdowns):

a. Deselect by index:

```
dropdown.deselectByIndex(index);
```

b. Deselect by visible text:

```
dropdown.deselectByVisibleText("Option Text");
```

c. Deselect by value attribute:

```
dropdown.deselectByValue("optionValue");
```

d. Deselect all options:

```
dropdown.deselectAll();
```

a. Get the first selected option:

```
WebElement selectedOption =  
dropdown.getFirstSelectedOption();
```

b. Get all selected options:

```
List<WebElement> selectedOptions =  
dropdown.getAllSelectedOptions();
```

5. Getting all options:

a. Get all options:

```
List<WebElement> options = dropdown.getOptions();
```

- The Select class provides these methods to interact with dropdown elements in Selenium WebDriver, allowing you to select options, deselect options, retrieve selected values, and get all available options within the dropdown.

II. Methods under Select Class

- The Select class in Selenium WebDriver provides various methods to interact with dropdown (select) elements on a web page.

Here are some commonly used methods of the Select class:

1. `selectByIndex(int index)`: Selects an option by its index. The index starts from 0 for the first option.
2. `selectByValue(String value)`: Selects an option by its value attribute.
3. `selectByVisibleText(String text)`: Selects an option by its visible text.

4. `deselectByIndex(int index)`: Deselects an option by its index.
 5. `deselectByValue(String value)`: Deselects an option by its value attribute.
 6. `deselectByVisibleText(String text)`: Deselects an option by its visible text.
 7. `deselectAll()`: Deselects all selected options in a multi-select dropdown.
 8. `getOptions()`: Returns a list of all available options in the dropdown.
 9. `getAllSelectedOptions()`: Returns a list of all selected options in a multi-select dropdown.
 10. `getFirstSelectedOption()`: Returns the first selected option in a multi-select dropdown.
- These methods allow you to select options, deselect options, retrieve selected values, and obtain information about the available options in a dropdown using the `Select` class in Selenium WebDriver.

III. Handling Dropdowns and Multiple Select Operations using Select Class

- To handle dropdowns and perform multiple select operations using the Select class in Selenium WebDriver, you can follow these steps:

1. Instantiate the Select class:

- a. First, create an instance of the Select class by locating the dropdown element on the web page.
- b. You can use various locator strategies such as ID, name, CSS selector, or XPath to identify the dropdown element

```
WebElement dropdownElement =  
driver.findElement(By.id("dropdownId"));  
Select dropdown = new Select(dropdownElement);
```

2. Select an option:

- a. To select an option from the dropdown, you can use one of the available methods provided by the Select class.
- b. You can select by index, value, or visible text.

```
// Select by index  
dropdown.selectByIndex(2);  
  
// Select by value  
dropdown.selectByValue("optionValue");  
  
// Select by visible text  
dropdown.selectByVisibleText("Option Text");
```

3. Deselect an option:

- a. If the dropdown allows multiple selections (i.e., it is a multi-select dropdown), you can deselect options using the deselect methods.

```
// Deselect by index
dropdown.deselectByIndex(2);

// Deselect by value
dropdown.deselectByValue("optionValue");

// Deselect by visible text
dropdown.deselectByVisibleText("Option Text");

// Deselect all options
dropdown.deselectAll();
```

- a. You can retrieve the selected options from a multi-select dropdown using the `getAllSelectedOptions()` method.

```
List<WebElement> selectedOptions =
dropdown.getAllSelectedOptions();
for (WebElement option : selectedOptions) {
    System.out.println(option.getText());
}
```

5. Get all options:

- a. To get all the available options in the dropdown, you can use the `getOptions()` method.

```
List<WebElement> allOptions = dropdown.getOptions();
for (WebElement option : allOptions) {
    System.out.println(option.getText());
}
```

- By using the Select class methods, you can interact with dropdown elements and perform various select and deselect operations based on your testing requirements.

IV. Actions Class and Action Interface

- The Actions class and Action interface in Selenium WebDriver are used for performing advanced user interactions, such as mouse actions, keyboard actions, drag-and-drop operations, and more.
- Let's explore the Actions class and Action interface in more detail:

1. Actions Class:

- a. The Actions class is part of the `org.openqa.selenium.interactions` package in Selenium WebDriver.
- b. It provides a high-level API for performing complex user interactions.

Here are some key methods provided by the Actions class:

1. `click(WebElement element)`: Performs a mouse click on the specified element.
2. `doubleClick(WebElement element)`: Performs a double-click action on the specified element.

3. `contextClick(WebElement element)`: Performs a right-click (context-click) action on the specified element.
4. `sendKeys(CharSequence... keys)`: Sends keyboard keys to the currently focused element.
5. `dragAndDrop(WebElement source, WebElement target)`: Drags the source element and drops it onto the target element.
6. `moveToElement(WebElement element)`: Moves the mouse to the middle of the specified element.
7. `build()`: Generates a composite Action object containing all the actions specified.

2. Action Interface:

- a. The Action interface represents a single user interaction or a sequence of interactions.
- b. It is part of the `org.openqa.selenium.interactions` package in Selenium WebDriver.
- c. The Actions class uses the Action interface to perform various actions.
- d. The Action interface defines the `perform()` method, which is used to execute the action or sequence of actions.

Here's an example that demonstrates the usage of the Actions class and Action interface to perform a drag-and-drop operation:

```
import org.openqa.selenium.interactions.Actions;
import org.openqa.selenium.WebDriver;
import org.openqa.selenium.WebElement;
import org.openqa.selenium.chrome.ChromeDriver;

public class DragAndDropExample {
    public static void main(String[] args) {
        // Set the path to the chromedriver executable
        System.setProperty("webdriver.chrome.driver",
```

```
// Create an instance of the Actions class
Actions actions = new Actions(driver);

// Perform the drag-and-drop action
actions.dragAndDrop(sourceElement,
targetElement).build().perform();

// Close the browser
driver.quit();
}
```

- In this example, we first create an instance of the Actions class and use the dragAndDrop() method to specify the source and target elements.
- We then use the build() method to generate the composite action and perform() method to execute the action.

- By using the Actions class and Action interface, you can perform various user interactions on web elements, making your test scripts more powerful and flexible.

V. Methods for handling Keyboard and Mouse based interactions

- In Selenium WebDriver, you can handle keyboard and mouse-based interactions using the Actions class.
- The Actions class provides methods for performing various keyboard and mouse actions.
- Here are some commonly used methods for handling keyboard and mouse interactions:

1. Keyboard Actions:

- i. `sendKeys(CharSequence... keys)`: Sends keyboard keys to the currently focused element. You can pass multiple keys as arguments, such as `Keys.ENTER`, `Keys.BACKSPACE`, or any other key value.
- ii. `keyDown(CharSequence key)`: Performs a key down action on the specified key.
- iii. `keyUp(CharSequence key)`: Performs a key up action on the specified key.

2. Mouse Actions:

- i. `click(WebElement element)`: Performs a mouse click on the specified element.
- ii. `doubleClick(WebElement element)`: Performs a double-click action on the specified element.

- iii. `contextClick(WebElement element)`: Performs a right-click (context-click) action on the specified element.
- iv. `moveToElement(WebElement element)`: Moves the mouse to the middle of the specified element.
- v. `dragAndDrop(WebElement source, WebElement target)`: Drags the source element and drops it onto the target element.
- vi. `dragAndDropBy(WebElement source, int xOffset, int yOffset)`: Drags the source element and drops it at the specified offset.

Here's an example that demonstrates the usage of keyboard and mouse actions in Selenium WebDriver:

```
import org.openqa.selenium.By;
import org.openqa.selenium.WebDriver;
import org.openqa.selenium.WebElement;
import org.openqa.selenium.chrome.ChromeDriver;
import org.openqa.selenium.interactions.Actions;
import org.openqa.selenium.Keys;

public class KeyboardAndMouseActionsExample {
    public static void main(String[] args) {
        // Set the path to the chromedriver executable
        System.setProperty("webdriver.chrome.driver",
            "path/to/chromedriver");

        // Create an instance of ChromeDriver
        WebDriver driver = new ChromeDriver();

        // Navigate to the web page
        driver.get("https://example.com");

        // Locate the input element
        WebElement inputElement =
            driver.findElement(By.id("inputElementId"));
    }
}
```

- In this example, we first create an instance of the Actions class and use the sendKeys() method to send text to an input element.
- We then use the sendKeys() method with the Keys.
- ENTER key to simulate pressing the Enter key.
- After that, we use the moveToElement() method to move the mouse to a button element and the click() method to perform a mouse click action.
- By using the Actions class and its methods, you can handle keyboard and mouse-based interactions in your Selenium WebDriver tests.

VI. Testing Drag and Drop and Mouse Hover functionality using Actions Class

- To test drag and drop and mouse hover functionality using the Actions class in Selenium WebDriver, you can use the following methods:

1. Drag and Drop:

```
import org.openqa.selenium.By;
import org.openqa.selenium.WebDriver;
import org.openqa.selenium.WebElement;
import org.openqa.selenium.chrome.ChromeDriver;
import org.openqa.selenium.interactions.Actions;

public class DragAndDropExample {
    public static void main(String[] args) {
        // Set the path to the chromedriver executable
        System.setProperty("webdriver.chrome.driver",

        // Create an instance of ChromeDriver
        WebDriver driver = new ChromeDriver();

        // Navigate to the web page
        driver.get("https://example.com");

        // Locate the source and target elements
        WebElement sourceElement =
driver.findElement(By.id("sourceElementId"));
        WebElement targetElement =
driver.findElement(By.id("targetElementId"));

        // Create an instance of the Actions class
        Actions actions = new Actions(driver);

        // Perform drag and drop action
        actions.dragAndDrop(sourceElement,
targetElement).perform();

        // Close the browser
        driver.quit();
    }
}
```

- In this example, we first locate the source element and target element using appropriate locators (such as `By.id()`).
- We then create an instance of the `Actions` class and use the `dragAndDrop()` method to perform the drag and drop action from the source element to the target element.

```
import org.openqa.selenium.By;
import org.openqa.selenium.WebDriver;
import org.openqa.selenium.WebElement;
import org.openqa.selenium.chrome.ChromeDriver;
import org.openqa.selenium.interactions.Actions;

public class MouseHoverExample {
    public static void main(String[] args) {
        // Set the path to the chromedriver executable
        System.setProperty("webdriver.chrome.driver",
"path/to/chromedriver");

        // Create an instance of ChromeDriver
        WebDriver driver = new ChromeDriver();

        // Navigate to the web page
        driver.get("https://example.com");

        // Locate the element to perform mouse hover
        WebElement elementToHover =
driver.findElement(By.id("elementToHoverId"));
```

- In this example, we locate the element on which we want to perform the mouse hover using an appropriate locator.
- We then create an instance of the Actions class and use the `moveToElement()` method to move the mouse to the specified element.
- Finally, we use the `perform()` method to execute the mouse hover action.
- By using the Actions class and its methods, you can easily test drag and drop and mouse hover functionality in your Selenium WebDriver tests.

5. Interacting with Web Elements Part-III

Information

I. What are Alerts ?

- In the context of web testing, an alert is a pop-up dialog box that appears on a webpage to provide important information or to prompt the user for some action.
- Alerts can be of different types:
 1. Simple Alerts: These are basic alerts that display a message and contain a single button to close the alert.
 2. Confirmation Alerts: These alerts provide a message along with two buttons: OK and Cancel. They are used when the user needs to confirm or cancel an action.
 3. Prompt Alerts: These alerts are similar to confirmation alerts but include an input field where the user can enter some text or value.
- When interacting with alerts in Selenium WebDriver, you can use the Alert interface provided by the WebDriver API.
- It offers methods to handle different types of alerts, such as accepting or dismissing an alert, retrieving the text of an alert, and entering values in prompt alerts.

II. Types of alerts

- In the context of web testing, there are generally three types of alerts that can appear on a webpage:

1. Simple Alerts:

- a. Simple alerts display a message with an "OK" button.
- b. They are used to provide information to the user or display an error message.
- c. Once the alert is displayed, the user can only dismiss it by clicking the "OK" button.

2. Confirmation Alerts:

- a. Confirmation alerts require the user to confirm or cancel an action.
- b. They display a message along with "OK" and "Cancel" buttons.
- c. The user can click "OK" to proceed with the action or click "Cancel" to abort it.
- d. Confirmation alerts are commonly used for actions that may have irreversible consequences, such as deleting data or submitting a form.

3. Prompt Alerts:

- a. Prompt alerts are similar to confirmation alerts but include an additional input field where the user can enter some text or value.
- b. They prompt the user to provide input or enter information.
- c. Prompt alerts typically have a message, an input field, and "OK" and "Cancel" buttons.
- d. The user can enter text in the input field and then choose to proceed or cancel the action.

- When automating tests using Selenium WebDriver, it is important to handle alerts appropriately based on their type.
- You can use WebDriver's Alert interface and its methods, such as `accept()`, `dismiss()`, and `sendKeys()`, to interact with and handle different types of alerts in your test scripts.

III. Handling Alerts in Selenium WebDriver

- To handle alerts in Selenium WebDriver, you can use the Alert interface provided by the WebDriver API.

Here are the steps to handle alerts:

1. Switch to the Alert: First, you need to switch the WebDriver's focus to the alert using the `switchTo().alert()` method.

```
Alert alert = driver.switchTo().alert();
```

2. Get the Alert Text: You can retrieve the text displayed in the alert using the `getText()` method.

```
String alertText = alert.getText();  
System.out.println("Alert Text: " + alertText);
```

3. Accept or Dismiss the Alert: Depending on the type of alert, you can accept it (click "OK") or dismiss it (click "Cancel") using the `accept()` or `dismiss()` method.

```
alert.accept(); // To accept the alert  
// or  
alert.dismiss(); // To dismiss the alert
```

4. Enter Input in Prompt Alert: If the alert is a prompt alert that requires user input, you can use the `sendKeys()` method to enter text in the input field of the alert.

```
alert.sendKeys("Input Value"); // Enter text in the input field
```

5. Switch back to the Default Content: After handling the alert, you can switch the WebDriver's focus back to the default content using the `defaultContent()` method.

```
driver.switchTo().defaultContent();
```

- It's important to note that alerts can occur in different contexts, such as the main window, frames, or pop-up windows.
- You may need to switch the WebDriver's focus accordingly using methods like `switchTo().defaultContent()` or `switchTo().frame()` before interacting with the alert.
- Make sure to handle alerts appropriately based on their type and the desired actions in your test scenarios.

IV. What is a Modal Dialog Box?

- A modal dialog box, also known as a modal window, is a graphical user interface (GUI) element that appears on top of the current window or screen and requires user interaction before allowing any other interactions with the underlying content.
- It is called "modal" because it creates a mode that restricts the user's interaction with the rest of the application until the dialog box is closed or a specific action is taken.
- Modal dialog boxes are commonly used to display important messages, request user input, or prompt for confirmation before proceeding with an action.
- They typically contain buttons or options for the user to make a choice or provide input.

The characteristics of a modal dialog box include:

1. Focus: It captures the user's attention by overlaying the content and bringing the dialog box into focus, making it the active element.
2. Blocking Interaction: It blocks interaction with the rest of the application or underlying content until the dialog box is closed or the required action is taken.
3. Modal Behavior: It enforces a mode where the user is expected to complete an action within the dialog box before interacting with other parts of the application.

- Modal dialog boxes are widely used in web applications, desktop applications, and mobile applications to ensure important messages or actions receive the user's immediate attention and prevent unintended actions.
- It's important to handle modal dialog boxes properly in automated tests to ensure the test flow is not disrupted.
- Selenium WebDriver provides methods to switch the focus to the modal dialog box and interact with its elements, similar to handling alerts.

V. Handling Multiple Windows and Multiple Tabs using Selenium WebDriver

- To handle multiple windows and multiple tabs using Selenium WebDriver, you can follow these steps:
 1. Get the window handles:
 - a. When a new window or tab is opened, you need to retrieve the window handles of all the open windows or tabs.
 - b. You can use the `driver.getWindowHandles()` method to get a set of all the window handles.
 2. Switch to a specific window or tab:
 - a. To switch the driver's focus to a specific window or tab, you need to use the `driver.switchTo().window()` method and pass the window handle as a parameter.

- b. You can loop through the window handles and switch to the desired window or tab based on your requirements.
- 3. Perform actions on the window or tab:
 - a. Once the driver's focus is on a specific window or tab, you can perform various actions such as interacting with elements, clicking buttons, or navigating to different URLs.
 - b. Use the WebDriver methods to interact with the elements on the currently active window or tab.
- 4. Switch back to the main window or tab:
 - a. After you have finished interacting with a specific window or tab, you can switch the driver's focus back to the main window or tab.
 - b. You can keep track of the main window or tab handle and use the `driver.switchTo().window()` method to switch back to it.

Here's an example code snippet demonstrating the handling of multiple windows and multiple tabs using Selenium WebDriver in Java:

```
// Get the window handles
Set<String> windowHandles = driver.getWindowHandles();

// Switch to a specific window or tab
for (String handle : windowHandles) {
    driver.switchTo().window(handle);
    // Perform actions on the current window or tab
    // ...
}

// Switch back to the main window or tab
String mainWindowHandle = driver.getWindowHandle();
driver.switchTo().window(mainWindowHandle);
```

- Note that the specific implementation may vary depending on your application's behavior and the actions you need to perform on each window or tab.
- Ensure that you use the correct window handles and handle any exceptions that may occur during the switching process.

VI. Scrolling on a web page using JavaScriptExecutor

- To scroll on a web page using JavaScriptExecutor in Selenium WebDriver, you can use the `executeScript()` method provided by the driver.

Here's an example code snippet in Java:

```
// Create an instance of JavaScriptExecutor
JavascriptExecutor js = (JavascriptExecutor) driver;

// Scroll vertically by pixels
js.executeScript("window.scrollBy(0, 500)");

// Scroll to a specific element
WebElement element = driver.findElement(By.id("elementId"));
js.executeScript("arguments[0].scrollIntoView();", element);

// Scroll to the bottom of the page
js.executeScript("window.scrollTo(0, document.body.scrollHeight);");
```

In the above code:

1. `window.scrollBy(x, y)` scrolls the page by x pixels horizontally and y pixels vertically.
2. `arguments[0].scrollIntoView()` scrolls the page to bring the specified element into the viewport.
3. `window.scrollTo(x, y)` scrolls the page to the specified coordinates.

You can adjust the scrolling behavior based on your requirements.

VII. Verifying Tooltip using Selenium WebDriver

- To verify a tooltip using Selenium WebDriver, you can use the `getAttribute()` method to retrieve the "title" attribute of the element that triggers the tooltip.

Here's an example code snippet in Java:

```
// Locate the element that triggers the tooltip
WebElement element = driver.findElement(By.id("elementId"));

// Get the tooltip text
String tooltipText = element.getAttribute("title");

// Compare the tooltip text with the expected value
String expectedTooltip = "Expected tooltip text";
if (tooltipText.equals(expectedTooltip)) {
    System.out.println("Tooltip text is correct: " + tooltipText);
} else {
    System.out.println("Tooltip text is incorrect. Expected: " +
        expectedTooltip + ", Actual: " + tooltipText);
}
```

- In the above code, replace "elementId" with the actual ID or locator of the element on which the tooltip is displayed.
- Then, retrieve the "title" attribute using `getAttribute("title")` and compare it with the expected tooltip text.
- If the actual tooltip text matches the expected value, it means the tooltip is correct.

Make sure you have appropriate wait conditions in place to ensure that the tooltip is fully loaded before retrieving its text.

VIII. What is an IFrame?

- An IFrame, short for Inline Frame, is an HTML element that allows you to embed another HTML document within the current document.
- It is like a window within a window, where you can display content from a different source or webpage within your main webpage.
- IFrames are commonly used to embed external content such as advertisements, maps, videos, or social media widgets into a webpage.
- They provide a way to include content from different domains or sources while maintaining the overall structure and layout of the main webpage.
- To interact with elements inside an IFrame using Selenium WebDriver, you need to switch the driver's

focus to the IFrame using the `switchTo().frame()` method.

- Once inside the IFrame, you can locate and interact with elements as you would normally do.

Here's an example of how to switch to an IFrame and perform actions inside it using Java:

```
// Switch to the IFrame by index (0 represents the first IFrame on the page)
driver.switchTo().frame(0);

// Now, you can interact with elements inside the IFrame
WebElement element = driver.findElement(By.id("elementId"));
element.sendKeys("Hello, World!");

// Switch back to the main page
driver.switchTo().defaultContent();
```

- In the above code, we first switch to the IFrame using the `switchTo().frame()` method and provide the index or the name/id of the IFrame.
- Then, we locate and interact with elements inside the IFrame.
- Finally, we switch back to the main page using `switchTo().defaultContent()`.

IX. Identifying an IFrame

To identify an IFrame in a web page, you can use various methods provided by Selenium WebDriver. Here are some common ways to identify an IFrame:

1. By Index:

- a. If the web page contains multiple IFrames, you can identify an IFrame using its index.
- b. The index starts from 0, so the first IFrame would have an index of 0, the second IFrame would have an index of 1, and so on.
- c. You can switch to an IFrame using the `switchTo().frame()` method and pass the index as an argument

```
// Switch to the first IFrame  
driver.switchTo().frame(0);
```

2. By Name or ID:

- a. If the IFrame has a specific name or ID attribute, you can switch to it by specifying the name or ID using the `switchTo().frame()` method.

```
// Switch to the IFrame by name  
driver.switchTo().frame("iframeName");
```

```
// Switch to the IFrame by ID  
driver.switchTo().frame("iframeId");
```

3. By WebElement:

- a. If you have a reference to the WebElement representing the IFrame, you can switch to it using the `switchTo().frame()` method and pass the WebElement as an argument.

```
// Locate the IFrame element
WebElement iframeElement =
driver.findElement(By.tagName("iframe"));

// Switch to the IFrame using the WebElement
driver.switchTo().frame(iframeElement);
```

- You can choose the appropriate method based on the specific attributes and structure of the web page you are working with.

X. Switching to IFrames using Selenium WebDriver

- To switch to an IFrame using Selenium WebDriver, you can use the `switchTo().frame()` method.

Here's an example of how to switch to an IFrame:

```
// Switch to an IFrame by index
driver.switchTo().frame(0);

// Switch to an IFrame by name or ID
driver.switchTo().frame("iframeName");

// Switch to an IFrame by WebElement
WebElement iframeElement =
driver.findElement(By.tagName("iframe"));
driver.switchTo().frame(iframeElement);
```

1. `driver.switchTo().frame(0)` - This code switches to the first IFrame on the web page using its index. The index starts from 0, so the first IFrame would have an index of 0.
 2. `driver.switchTo().frame("iframeName")` - This code switches to an IFrame with a specific name or ID. Replace "iframeName" with the actual name or ID of the IFrame you want to switch to.
 3. `WebElement iframeElement = driver.findElement(By.tagName("iframe"))` - This code locates an IFrame element using the `findElement()` method and the `By.tagName()` locator strategy. You can use other locator strategies like `By.id()`, `By.name()`, etc., depending on the attributes of the IFrame element.
 4. `driver.switchTo().frame(iframeElement)` - This code switches to the IFrame using the `WebElement` representing the IFrame.
- After switching to the IFrame, you can interact with the elements inside the IFrame using regular WebDriver methods.
 - To switch back to the default content (main web page), you can use `driver.switchTo().defaultContent()`.

6. Robot Class to handle window operations

Information

I. Introduction to Robot class

- The Robot class and the WebDriver framework are separate concepts and not directly related.
- The Robot class is part of the Java AWT (Abstract Window Toolkit) library and is used for simulating user interactions at the system level, while WebDriver is a framework for automating web browsers.
- The Robot class is primarily used for tasks that involve simulating keyboard input, mouse clicks, and mouse movements at the system level, such as automating desktop applications or controlling the mouse and keyboard outside of a browser context.
- On the other hand, WebDriver is specifically designed for automating web browsers.
- It provides a set of APIs and methods to interact with web elements, navigate through web pages, and perform various actions on a web application.
- WebDriver supports different browser drivers (such as ChromeDriver, FirefoxDriver, etc.) and provides a standardized way to automate web testing.
- Therefore, if you are working with Selenium WebDriver, you would typically use WebDriver's built-in methods and APIs to automate web browser interactions, rather than relying on the Robot class.
- WebDriver offers more specific and robust functionality for web automation tasks.

- In summary, while the Robot class is useful for automating tasks at the system level, WebDriver is the preferred choice for web browser automation.

II. Methods under Robot class (KeyPress, Key Release, Mouse Press, Mouse Release, Mouse Move)

- The methods under the Robot class in Java AWT (Abstract Window Toolkit) provide capabilities for simulating user interactions at the system level.
- While these methods can be used independently of Selenium WebDriver, they can still be used in combination with WebDriver to enhance the automation capabilities.

1. KeyPress and KeyRelease:

- The keyPress method is used to simulate pressing a key on the keyboard.
- The keyRelease method is used to simulate releasing a key that was previously pressed.
- These methods can be used to simulate keyboard actions such as typing text, pressing function keys, or triggering keyboard shortcuts.

2. MousePress and MouseRelease:

- The mousePress method is used to simulate pressing a mouse button.
- The mouseRelease method is used to simulate releasing a previously pressed mouse button.

- These methods can be used to simulate mouse clicks, including left-click, right-click, or middle-click.

3. MouseMove:

- The mouseMove method is used to move the mouse pointer to a specific screen coordinate.
- This method can be used to simulate mouse movements or position the mouse over a particular element on the screen.
- When using WebDriver in combination with the Robot class, you can leverage the Robot class methods to perform system-level interactions, such as entering text using keyPress and keyRelease, clicking on elements using mousePress and mouseRelease, or moving the mouse pointer using mouseMove.
- However, it's important to note that using the Robot class for browser automation may have limitations compared to WebDriver's built-in methods.
- WebDriver provides more granular control and visibility into the web page's elements and behavior, making it the preferred choice for automating web browser interactions.
- It's recommended to use the Robot class sparingly in combination with WebDriver, only when specific system-level interactions are required that cannot be achieved solely with WebDriver's methods.

III. Implementation of the Robot class

- To implement the Robot class in Selenium WebDriver, you need to import the necessary packages and create an instance of the Robot class.

Here's an example of how to use the Robot class to perform keyboard and mouse actions:

```
import org.openqa.selenium.WebDriver;
import org.openqa.selenium.chrome.ChromeDriver;
import java.awt.AWTException;
import java.awt.Robot;
import java.awt.event.KeyEvent;
import java.awt.event.InputEvent;

public class RobotExample {
    public static void main(String[] args) {
        // Set the system property for the WebDriver
        System.setProperty("webdriver.chrome.driver",
"path/to/chromedriver");

        // Create a new instance of the WebDriver
        WebDriver driver = new ChromeDriver();

        // Open a webpage
        driver.get("https://www.example.com");

        try {
            // Create an instance of the Robot class
            Robot robot = new Robot();

            // Simulate pressing 'Ctrl' + 'A' to select all text
            robot.keyPress(KeyEvent.VK_CONTROL);
            robot.keyPress(KeyEvent.VK_A);
            robot.keyRelease(KeyEvent.VK_A);
            robot.keyRelease(KeyEvent.VK_CONTROL);

            // Simulate pressing 'Ctrl' + 'C' to copy the selected text
            robot.keyPress(KeyEvent.VK_CONTROL);
            robot.keyPress(KeyEvent.VK_C);
            robot.keyRelease(KeyEvent.VK_C);
            robot.keyRelease(KeyEvent.VK_CONTROL);
        } catch (AWTException e) {
            e.printStackTrace();
        }
    }
}
```

```
// Simulate clicking the right mouse button
robot.mousePress(InputEvent.BUTTON3_DOWN_MASK);
robot.mouseRelease(InputEvent.BUTTON3_DOWN_MASK);

// Simulate moving the mouse pointer to a specific coordinate
robot.mouseMove(500, 500);

} catch (AWTException e) {
    e.printStackTrace();
}

// Close the browser
driver.quit();
}
```

- In this example, we first import the required packages for WebDriver, Robot, KeyEvent, and InputEvent.
- We then create an instance of the WebDriver (in this case, ChromeDriver) and open a webpage.
- Inside the try-catch block, we create an instance of the Robot class and use its methods to simulate keyboard and mouse actions.
- We use `keyPress` and `keyRelease` to simulate pressing and releasing keys, and `mousePress` and `mouseRelease` to simulate mouse button clicks.

- We can also use `mouseMove` to move the mouse pointer to a specific coordinate on the screen.
- Finally, we close the browser using `driver.quit()`.
- Note that for this example to work, you need to replace "path/to/chromedriver" with the actual path to the ChromeDriver executable on your system.
- Please ensure that you use the Robot class with caution and only when necessary, as it simulates system-level interactions and can have side effects outside the scope of the web browser.

IV. Advantages and disadvantages of the Robot class

- The Robot class in WebDriver provides a way to simulate keyboard and mouse actions at the system level. It can be useful in certain scenarios, but it also has its advantages and disadvantages.

Here are some of them:

Advantages of the Robot class:

1. Simulating real user interactions: The Robot class allows you to mimic user actions such as pressing keys, clicking the mouse, or moving the mouse pointer. This can be helpful in automating test scenarios that involve complex user interactions.
2. System-level actions: The Robot class operates at the system level, which means it can perform actions outside the browser window. It can interact with

other applications or perform tasks that are not directly accessible through WebDriver.

3. Cross-platform compatibility: The Robot class is part of the Java AWT (Abstract Window Toolkit), which makes it platform-independent. It can be used on different operating systems without requiring specific platform-specific configurations.

Disadvantages of the Robot class:

1. Lack of browser context: The Robot class operates at the system level and does not have direct access to the browser's internal structure or elements. It cannot interact with web elements like WebDriver does. Therefore, it may not be suitable for testing scenarios that require precise control over web elements.
2. Fragility and maintenance: Since the Robot class relies on simulating user actions at the system level, it is more prone to issues related to system configuration and environment. Any changes in the system or the application being tested may require adjustments or updates to the Robot-based code.
3. Limited browser compatibility: The Robot class's actions are not specific to any particular browser, which means they may not be fully compatible with all browsers. Some browser-specific features or behaviors may not be accurately replicated using the Robot class.

4. Lack of synchronization: The Robot class does not have built-in synchronization mechanisms to wait for elements or handle dynamic changes on a web page. It performs actions immediately without considering the state of the application, which can lead to synchronization issues in test scenarios.
- It's important to consider these advantages and disadvantages when deciding whether to use the Robot class in WebDriver.
- While it can be a useful tool for specific use cases, it may not be suitable for all testing scenarios and should be used judiciously.

V. Limitations of the Robot class

- The Robot class in WebDriver provides a way to simulate keyboard and mouse actions at the system level.

7. TestNG Framework

Information

I. Introduction to TestNG

- TestNG is a testing framework for Java that is widely used in the Selenium WebDriver ecosystem.

- It provides a powerful and flexible platform for organizing and running tests, generating reports, and managing test configurations.
- TestNG enhances the capabilities of Selenium WebDriver by introducing features such as parallel test execution, test dependencies, data-driven testing, and test configuration management.

1. Test Organization:

- a. TestNG allows you to organize your tests into test suites, test classes, and test methods, making it easy to manage and execute specific sets of tests.

2. Annotations:

- a. TestNG uses annotations to define the behavior of test methods. Annotations such as `@Test`, `@BeforeMethod`, `@AfterMethod`, and more provide precise control over test execution and setup/teardown operations.

3. Parallel Execution:

- a. TestNG supports parallel execution of tests, allowing you to run tests concurrently on multiple threads or across multiple machines.
- b. This can significantly reduce test execution time and improve efficiency.

4. Test Dependencies:

- a. TestNG allows you to specify dependencies between test methods, ensuring that certain tests are executed only after the successful execution of their dependencies.
- b. This enables better control over test execution order and helps in managing complex test scenarios.

5. Data-Driven Testing:

- a. TestNG supports data-driven testing by providing mechanisms to execute the same test method with different sets of data.
- b. This is useful for testing different input combinations or scenarios.

6. Test Configuration Management:

- a. TestNG allows you to define and manage test configurations using configuration methods and parameters.
- b. This enables you to define and reuse common setup and teardown operations across multiple tests.

7. Reporting and Logging:

- a. TestNG provides comprehensive test result reporting, including information about passed, failed, and skipped tests.
- b. It also supports custom reporting and integration with popular build tools and continuous integration (CI) systems.

- To use TestNG with Selenium WebDriver, you need to set up your test project with TestNG dependencies and configure your test classes and methods using TestNG annotations.
- You can then execute your tests using TestNG test runners, generate test reports, and leverage the advanced features offered by TestNG to enhance your test automation process.
- Overall, TestNG is a valuable tool for Selenium WebDriver test automation, providing enhanced features and capabilities that make test development, execution, and management more efficient and flexible.

II. Installing TestNG Plug-in in Eclipse

- To install the TestNG plugin in Eclipse for WebDriver, you can follow these steps:
 1. Launch Eclipse: Open the Eclipse IDE on your computer.
 2. Go to Eclipse Marketplace: Click on the "Help" menu in the toolbar and select "Eclipse Marketplace" from the dropdown.
 3. Search for TestNG: In the Eclipse Marketplace dialog, type "TestNG" in the search bar located in the top-right corner.

4. Select TestNG: From the search results, you should see the "TestNG for Eclipse" plugin. Click on the "Go to Marketplace" button next to it.
 5. Install TestNG: On the TestNG for Eclipse page in the Eclipse Marketplace, click the "Install" button.
 6. Review the installation details: In the installation dialog, you will see the list of features that will be installed. Review the details and click the "Confirm" button.
 7. Accept the license terms: On the next page, you will be presented with the license terms. Read and accept the terms, then click the "Finish" button.
 8. Restart Eclipse: After the installation is complete, Eclipse will prompt you to restart the IDE. Click "Yes" to restart Eclipse.
 9. Verify TestNG installation: Once Eclipse restarts, you can verify the successful installation of TestNG by creating a new Java project or opening an existing one. Right-click on the project, go to "Properties" and check if "TestNG" appears in the left sidebar.
- With the TestNG plugin installed, you can now create TestNG test classes, add TestNG annotations to your test methods, and execute TestNG tests in Eclipse.
 - Note: Make sure you have the appropriate internet connection to download and install plugins from the Eclipse Marketplace.

III. Rules to write TestNG

- When writing TestNG tests for WebDriver, it's important to follow certain rules and best practices to ensure efficient and effective test automation.
1. Use TestNG annotations:
 - a. TestNG provides annotations that help in structuring and organizing test cases. Use annotations like `@Test`, `@BeforeMethod`, `@AfterMethod`, `@BeforeClass`, and `@AfterClass` to define test methods and setup/teardown methods.
 2. Separate test logic from setup/teardown:
 - a. Keep your test methods focused on the specific test case, and move common setup and teardown code to separate methods using TestNG annotations like `@BeforeMethod` and `@AfterMethod`.
 3. Use assertions for verification:
 - a. Use TestNG's assertion methods (`Assert.assertEquals()`, `Assert.assertTrue()`, etc.) to validate expected results against actual results during the test execution.
 4. Parameterize tests:
 - a. TestNG allows parameterization of test methods using data providers.

- b. This helps in executing the same test with different input data.
 - c. Utilize the `@DataProvider` annotation to provide data for parameterized tests.
- 5. Use TestNG's test configuration options:
 - a. TestNG provides various configuration options such as parallel execution, grouping, and dependency management.
 - b. Use these options to customize the test execution and improve test efficiency.
- 6. Keep tests independent and isolated:
 - a. Each test method should be independent of other tests and not rely on the state or results of previous tests.
 - b. Isolate the test cases to prevent interference and ensure reliable test execution.
- 7. Use proper naming conventions:
 - a. Follow a consistent and descriptive naming convention for test methods, making it easier to understand the purpose of each test.
- 8. Use meaningful test outputs and logging:
 - a. Utilize logging frameworks like Log4j or the built-in `System.out.println()` statements to log useful information during test execution.
 - b. This helps in debugging and troubleshooting.
- 9. Utilize TestNG's reporting features:

- a. TestNG provides built-in HTML and XML reports that give detailed information about test execution.
 - b. Enable and utilize these reports to get insights into test results and failures.
10. Regularly review and refactor tests:
 - a. As your test suite grows, review and refactor tests to maintain code cleanliness, readability, and maintainability.
- By following these rules, you can write well-structured, maintainable, and reliable TestNG tests for your WebDriver automation projects.

II. Annotations

- Annotations play a crucial role in TestNG as they define the behavior and execution flow of test methods.

1. @Test:

- a. This annotation is used to mark a method as a test method.
- b. TestNG will consider any method annotated with @Test for test execution.

Example:

```
@Test
public void testLogin() {
    // Test logic here
}
```

2. @BeforeMethod:

- a. This annotation is used to mark a method that should be executed before each test method.
- b. It is typically used for setting up preconditions or initializing resources required for the test.

Example:

```
@BeforeMethod
public void setUp() {
    // Setup logic here
}
```

3. @AfterMethod:

- a. This annotation is used to mark a method that should be executed after each test method.
- b. It is used for cleaning up resources or performing post-test actions.

Example:

```
@AfterMethod
public void tearDown() {
    // Teardown logic here
}
```

4. @BeforeClass:

- a. This annotation is used to mark a method that should be executed once before all the test methods in a class.
- b. It is often used for one-time setup tasks.

Example:

```
@BeforeClass
public static void init() {
    // One-time setup logic here
}
```

5. @AfterClass:

- a. This annotation is used to mark a method that should be executed once after all the test methods in a class have run.
- b. It is typically used for one-time cleanup tasks.

Example:

```
@AfterClass
public static void cleanUp() {
    // One-time cleanup logic here
}
```

- These annotations provide control over the test execution flow and enable proper setup and teardown of resources.
- By using them effectively, you can structure your test code and manage test dependencies efficiently.

III. Grouping

- In TestNG, you can group your test methods using the groups attribute in the @Test annotation. Grouping allows you to categorize your tests and execute them selectively based on the specified groups.

Explanation:

- Grouping in TestNG allows you to organize your tests based on different criteria, such as functional areas, priority levels, or test types.
- By assigning test methods to specific groups, you can easily run tests that belong to a particular group or exclude certain groups from execution.

Example:

- Suppose you have two test methods testLogin and testRegistration, and you want to group them based on their functionality.
- You can define two groups, such as "Login" and "Registration", and assign the test methods to their respective groups.

```
@Test(groups = "Login")
public void testLogin() {
    // Test logic for login
}

@Test(groups = "Registration")
public void testRegistration() {
    // Test logic for registration
}
```

- In the above example, the testLogin method is assigned to the "Login" group, and the testRegistration method is assigned to the "Registration" group.
- To run tests based on groups, you can use the groups attribute in the TestNG XML configuration file or specify the groups directly when executing the tests programmatically.

TestNG XML File Structure

```
<?xml version="1.0" encoding="UTF-8"?>
<!DOCTYPE suite SYSTEM "https://testng.org/testng-1.0.dtd">
<suite name="Sample Suite">
  <test name="TestGroup1">
    <classes>
      <class name="com.example.TestClass1" />
      <class name="com.example.TestClass2" />
    </classes>
  </test>
  <test name="TestGroup2">
    <classes>
      <class name="com.example.TestClass3" />
    </classes>
  </test>
</suite>
```

For example, to run tests belonging to the "Login" group:

- TestNG XML configuration:

```
<test name="Login Tests">
  <groups>
    <run>
      <include name="Login"/>
    </run>
  </groups>
```

- Programmatic execution:

```
TestNG testng = new TestNG();  
testng.setGroups("Login");  
testng.setTestClasses(new Class[] { YourTestClass.class });  
testng.run();
```

- By grouping your tests, you can easily control and execute specific sets of tests based on their functionality or any other criteria that you define.

IV. Sequencing

- In TestNG, you can control the sequencing or order of execution of your test methods using the `dependsOnMethods` attribute in the `@Test` annotation.
- This allows you to specify dependencies between test methods, ensuring that certain tests are executed before others.

Explanation:

- Sequencing in TestNG allows you to define the order in which your test methods should be executed.

- By using the `dependsOnMethods` attribute, you can specify that a test method depends on the successful execution of one or more other test methods.
- This ensures that the dependent test method(s) are executed before the current test method.

Example:

- Suppose you have three test methods: `testLogin`, `testCreateAccount`, and `testLogout`.
- You want to ensure that the login test is executed first, followed by the create account test, and finally the logout test.
- You can specify the dependencies using the `dependsOnMethods` attribute.

```
@Test
public void testLogin() {
    // Test logic for login
}

@Test(dependsOnMethods = "testLogin")
public void testCreateAccount() {
    // Test logic for creating an account
}

@Test(dependsOnMethods = "testCreateAccount")
public void testLogout() {
    // Test logic for logout
}
```

- In the above example, the testLogin method is marked as a dependency for the testCreateAccount method, and the testCreateAccount method is marked as a dependency for the testLogout method.
- This ensures that the tests are executed in the specified order.
- Note that the method names specified in the dependsOnMethods attribute must match the actual method names of the dependent tests.
- By sequencing your test methods, you can control the order of execution and ensure that tests that depend on each other are executed in the desired sequence.

V. Parameterization

- Parameterization in TestNG allows you to pass parameters to your test methods, enabling you to run the same test logic with different input values.
- This is useful when you want to test different scenarios or variations of a test case without duplicating code.

Explanation:

- TestNG supports parameterization through the use of the @Parameters and @DataProvider annotations.
- The @Parameters annotation is used to define the parameters at the test suite level, while the @DataProvider annotation is used to provide the actual values for the parameters.

- By specifying the parameters in the test suite and providing values through a data provider, you can run the same test method with different inputs.

Example:

- Suppose you have a test method `testLogin` that takes two parameters: `username` and `password`.
- You want to run this test method with different sets of `username` and `password` values.

Here's an example of how you can achieve this using TestNG:

```
@Test
@Parameters({"username", "password"})
public void testLogin(String username, String password) {
    // Test logic for login using the provided username and
    // password
    // ...
}

@DataProvider(name = "credentials")
public Object[][] provideCredentials() {
    return new Object[][] {
        {"user1", "password1"},
        {"user2", "password2"},
        {"user3", "password3"}
    };
}
```

- In the above example, the `@Parameters` annotation is used to specify the names of the parameters

("username" and "password") in the test method signature.

- The `@DataProvider` annotation is used to define a data provider method `provideCredentials()` that returns a two-dimensional array of objects representing the different sets of username and password values.
- In your test suite XML file, you need to define the parameters and provide the values from the data provider:

```
<suite name="Test Suite">
  <test name="Login Test">
    <parameters>
      <parameter name="username" value="user1" />
      <parameter name="password" value="password1" />
    </parameters>
    <classes>
      <class name="com.example.LoginTests" />
    </classes>
  </test>
</suite>
```

- In this example, the `testLogin` method will be executed three times with different sets of username and password values provided by the `provideCredentials()` data provider.
- By parameterizing your test methods, you can easily run them with different input values, enabling you to test various scenarios and data combinations without duplicating code.

VI. Asserstions/verifications

- Assertions or verifications in WebDriver are used to verify the expected behavior or state of web elements or conditions during test execution.
- They allow you to validate whether certain conditions are true or false, and if they are not as expected, the test will fail.

Here's an explanation of assertions/verifications in WebDriver and an example:

1. Explanation of Assertions/Verifications:
 - a. WebDriver provides various assertion methods to perform verifications.
 - b. Some commonly used assertion methods are:
 - `assertEquals(expected, actual)`: Compares the expected and actual values for equality.
 - `assertTrue(condition)`: Checks whether the given condition is true.
 - `assertFalse(condition)`: Checks whether the given condition is false.
2. Example of Assertions/Verifications:

Here's an example that demonstrates the usage of assertions in WebDriver:

```
import org.openqa.selenium.By;
import org.openqa.selenium.WebDriver;
import org.openqa.selenium.WebElement;
import org.openqa.selenium.chrome.ChromeDriver;
import org.testng.Assert;
import org.testng.annotations.*;
```

```
@BeforeMethod
public void setUp() {
    // Set up the WebDriver instance
    System.setProperty("webdriver.chrome.driver",
"path/to/chromedriver");
    driver = new ChromeDriver();
    driver.get("https://example.com");
}

@AfterMethod
public void tearDown() {
    // Quit the WebDriver instance
    driver.quit();
}

@Test
public void testElementVisibility() {
    // Find an element on the page
    WebElement element = driver.findElement(By.id("my-
element"));

    // Verify that the element is visible
    Assert.assertTrue(element.isDisplayed(), "Element is not
visible.");
}

@Test
public void testPageTitle() {
    // Get the current page title
    String pageTitle = driver.getTitle();
```

- In this example, the `testElementVisibility` method verifies that a specific element with the id "my-element" is visible on the page using `assertTrue(element.isDisplayed())`.
- If the element is not visible, the test will fail and provide the specified failure message.
- The `testPageTitle` method verifies that the page title is equal to the expected title using `assertEquals(pageTitle, "Expected Page Title")`.
- If the page title does not match the expected title, the test will fail and display the specified failure message.

You can add more test methods with assertions based on your specific verification requirements.

VII. Test Suite

- A test suite in WebDriver is a collection of test cases that are grouped together to execute as a single unit.

- It allows you to organize and manage multiple test cases, define the order of their execution, and generate consolidated test reports.

Here's an explanation of test suites in WebDriver and an example:

1. Explanation of Test Suite:

- a. A test suite in WebDriver is typically created using a test framework like TestNG or JUnit.
- b. It is defined as a class or configuration file that specifies the set of test cases to be executed together.
- c. Test suites provide a convenient way to manage and execute multiple tests in a specific order.

Example of Test Suite:

Here's an example that demonstrates the usage of a test suite in WebDriver using TestNG:

```
import org.testng.annotations.*;

public class TestSuiteExample {
    @BeforeSuite
    public void setUp() {
        // Set up any preconditions for the test suite
        System.out.println("Setting up the test suite...");
    }

    @AfterSuite
    public void tearDown() {
        // Clean up or perform any post-test suite operations
        System.out.println("Tearing down the test suite...");
    }
}
```

```
@Test
public void testCase2() {
    // Test case logic for testCase2
    System.out.println("Executing testCase2...");
}

// More test methods...
}
```

- In this example, the TestSuiteExample class represents the test suite.
- The @BeforeSuite annotated method is executed before any test case in the suite, allowing you to set up any preconditions or configurations required for the entire suite.
- The @AfterSuite annotated method is executed after all the test cases in the suite have been executed.
- It is used for cleaning up or performing any post-test suite operations.
- The @Test annotated methods represent individual test cases that are part of the suite.

- You can add as many test methods as needed.
- To execute this test suite, you would run the test suite class using a testing framework such as TestNG.
- The framework will execute the test methods in the defined order and provide the test results and reports.
- Note that the actual test case logic is not included in this example for brevity.
- In a real scenario, you would include the necessary WebDriver code to interact with the web application and perform the desired test actions.
- By organizing test cases into a test suite, you can easily manage and execute them as a cohesive unit, making it convenient to run a specific set of tests or the entire suite.

VIII. Parallel/Cross Browser Testing

- Parallel/Cross Browser Testing in WebDriver allows you to execute your test cases simultaneously across multiple browsers or browser instances, improving test execution time and increasing test coverage.

Here's an explanation of Parallel/Cross Browser Testing in WebDriver and an example:

1. Explanation of Parallel/Cross Browser Testing:
 - a. Parallel testing involves running multiple tests or test cases concurrently, allowing you to execute tests in parallel threads or processes.

- b. Cross browser testing involves running the same test case on different web browsers to ensure compatibility and consistency across different browser environments.
- c. Parallel/Cross Browser Testing in WebDriver helps validate the behavior of your application across multiple browsers and speeds up the overall testing process.

2. Example of Parallel/Cross Browser Testing in WebDriver:

Here's an example that demonstrates how to perform Parallel/Cross Browser Testing using WebDriver:

```
import org.openqa.selenium.WebDriver;
import org.openqa.selenium.chrome.ChromeDriver;
import org.openqa.selenium.firefox.FirefoxDriver;
import org.openqa.selenium.remote.DesiredCapabilities;
import org.testng.annotations.*;

public class ParallelCrossBrowserTest {
    private WebDriver driver;

    @BeforeMethod
    @Parameters("browser")
    public void setUp(String browser) {
        // Set up WebDriver instance based on the specified browser
        if (browser.equalsIgnoreCase("chrome")) {
            System.setProperty("webdriver.chrome.driver",
"path/to/chromedriver");
            driver = new ChromeDriver();
        } else if (browser.equalsIgnoreCase("firefox")) {
```

```
@AfterMethod
public void tearDown() {
    // Close WebDriver instance
    driver.quit();
}

@Test
public void testExample() {
    // Test case logic using WebDriver methods
    driver.get("https://example.com");
    // Perform actions and assertions on the page
}

// More test methods...
}
```

- In this example, the `ParallelCrossBrowserTest` class represents the test class for parallel/cross browser testing.
- The `@BeforeMethod` annotated method is parameterized with the "browser" parameter, which specifies the browser to be used for each test method execution.
- The browser parameter is passed from the TestNG XML configuration file.
- Inside the `setUp` method, the `WebDriver` instance is initialized based on the specified browser.
- In this example, Chrome and Firefox browsers are used, but you can add more browser options as needed.
- The `@AfterMethod` annotated method is used to close the `WebDriver` instance after each test method execution.
- The `@Test` annotated methods represent the actual test cases.
- Inside these methods, you can write the test case logic using `WebDriver` methods to interact with the web application under test.
- To execute parallel/cross browser testing, you would configure the TestNG XML file with the desired browsers and test classes, and then execute the test suite.
- TestNG will create separate threads or processes to execute the test methods in parallel across the specified browsers.

- By leveraging parallel/cross browser testing in WebDriver, you can significantly reduce test execution time and ensure your application behaves consistently across different browsers, improving the overall quality of your software.

IX. Introduction to Reports

- Reports in WebDriver refer to generating detailed test execution reports that provide insights into the test results, including the status of each test case, the number of passed and failed tests, and any errors or exceptions encountered during test execution.
- These reports help in analyzing the test results, identifying issues, and facilitating communication among team members.

Here's an explanation of reports in WebDriver and an example:

1. Explanation of Reports in WebDriver:
 - a. Reports in WebDriver are typically generated using third-party libraries or frameworks that provide enhanced reporting capabilities beyond the default reporting features offered by WebDriver itself.
 - b. These libraries/frameworks help in generating comprehensive HTML, XML, or PDF reports that are easy to read and understand.

- Reports typically include information such as test case names, test status (passed/failed), execution time, error messages (if any), and screenshots (in case of failures).
- They also provide features like test categorization, grouping, and filtering to help organize and analyze test results effectively.

2. Example of Reports in WebDriver:

Here's an example that demonstrates generating a basic test execution report using the TestNG framework:

```
import org.testng.ITestResult;
import org.testng.annotations.*;
import org.testng.reporters.XMLReporter;
import org.testng.reporters.XMLSuiteResultWriter;
import org.testng.reporters.XMLTestResultWriter;

public class TestExecutionReport {

    @BeforeSuite
    public void setupReporters() {
        // Configure XML reporters for generating the test execution
        report
        XMLReporter xmlReporter = new XMLReporter();
        XMLSuiteResultWriter suiteResultWriter = new
XMLSuiteResultWriter();
        XMLTestResultWriter testResultWriter = new
XMLTestResultWriter();

        // Set the output directory for the reports
        String reportDirectory = "path/to/report/directory";
        xmlReporter.setOutputDirectory(reportDirectory);
        suiteResultWriter.setOutputDirectory(reportDirectory);
        testResultWriter.setOutputDirectory(reportDirectory);
    }
}
```

```
@Test
    public void testExample() {
        // Test case logic
    }

    // More test methods...

    @AfterSuite
    public void generateReport() {
        // Generate the final test execution report
        TestNG.getDefault().generateReports();
    }
}
```

- In this example, the TestExecutionReport class represents the test class that includes the test methods and report configuration.

- The `@BeforeSuite` annotated method is used to set up the XML reporters (XMLReporter, XMLSuiteResultWriter, XMLTestResultWriter) and configure the output directory for the reports.
- The `@AfterMethod` annotated method is used to handle the test result after each test method execution.
- In this example, it captures screenshots for failed tests, but you can perform other necessary actions as required.
- The `@Test` annotated methods represent the actual test cases. Inside these methods, you can write the test case logic.
- The `@AfterSuite` annotated method is used to generate the final test execution report using the `generateReports()` method provided by TestNG.
- This will generate the report in the specified output directory.
- By configuring and utilizing reporting frameworks like TestNG, ExtentReports, or Allure, you can generate comprehensive test execution reports with rich features and insights, making it easier to analyze and communicate the test results effectively.

X. Types of Reports

- There are several types of reports that can be generated in WebDriver to provide different levels of detail and insights into the test execution.

Here are some commonly used types of reports:

1. HTML Reports:

- a. HTML reports are widely used in WebDriver to generate detailed and interactive reports.
- b. These reports usually include test case names, test status (pass/fail), execution time, error messages (if any), screenshots (in case of failures), and other relevant information.
- c. HTML reports can be easily viewed in web browsers, making it convenient to navigate through test results and analyze them.

2. XML Reports:

- a. XML reports provide a structured format for storing test execution data.
- b. These reports are machine-readable and can be easily parsed by tools or frameworks for further analysis or integration with other systems.
- c. XML reports can be generated using libraries like TestNG, JUnit, or custom XML writers.

3. PDF Reports:

- a. PDF reports are commonly used when there is a need to generate printable and shareable reports.
- b. These reports provide a fixed layout and formatting, making them suitable for formal documentation and communication.
- c. PDF reports can be generated using libraries like Apache PDFBox or iText.

4. CSV Reports:

- a. CSV (Comma-Separated Values) reports provide a simple and lightweight format for storing test execution data in a tabular form.
- b. These reports can be easily imported into spreadsheets or databases for further analysis or processing.
- c. CSV reports are useful when you need to extract specific data points or perform statistical analysis on test results.

5. Dashboard Reports:

- a. Dashboard reports provide a summarized view of test execution results, highlighting key metrics and trends.
- b. These reports often include charts, graphs, and visual representations to provide a quick overview of test coverage, pass/fail ratios, and other important metrics.
- c. Dashboard reports can be generated using specialized reporting frameworks like ExtentReports or Allure.

Example:

Here's an example of generating an HTML report using the TestNG framework in WebDriver:

```
import org.testng.annotations.*;
import org.testng.reporters.HtmlReporter;
import org.testng.xml.XmlSuite;

public class TestExecutionReport {

    @BeforeSuite
```

```
@Test
    public void testExample() {
        // Test case logic
    }

    // More test methods...

    @AfterSuite
    public void generateReport() {
        // Generate the final HTML test execution report

        TestNG.getDefault().generateReport(XmlSuite.DEFAULT_SUITE_NAME);
    }
}
```

- In this example, the `TestExecutionReport` class represents the test class that includes the test methods and report configuration.
- The `@BeforeSuite` annotated method is used to set up the HTML reporter (`HtmlReporter`) and configure the output directory for the report.
- The `@Test` annotated methods represent the actual test cases. Inside these methods, you can write the test case logic.
- The `@AfterSuite` annotated method is used to generate the final HTML test execution report using the `generateReport()` method provided by TestNG.
- This will generate the report in the specified output directory.
- By utilizing the appropriate reporting frameworks and configuring the necessary reporters, you can generate different types of reports based on your requirements and preferences.

XI. Introduction to Extent Reports

- Extent Reports is a popular reporting framework for WebDriver that provides rich and interactive reports for test automation.
- It is built on top of the TestNG and JUnit frameworks and offers a comprehensive set of features to generate detailed and visually appealing reports.

Here are some key features of Extent Reports:

1. Interactive Reports:

- a. Extent Reports generates HTML reports that are highly interactive and user-friendly.
- b. The reports include detailed information about test execution, such as test case names, test status (pass/fail), execution time, error messages, screenshots (in case of failures), and more.
- c. The reports are designed to be visually appealing and provide a seamless navigation experience.

2. Dashboard View:

- a. Extent Reports provides a dashboard view that presents a summary of test execution results in the form of charts and graphs.
- b. The dashboard view gives a quick overview of key metrics like test status, pass/fail ratios, and test coverage. This helps stakeholders to get an overall understanding of the test execution outcome.

3. Test Categorization:

- a. Extent Reports allows you to categorize your tests into different groups, suites, or categories based on your project's needs.
- b. This helps in organizing and structuring your test cases, making it easier to analyze and report on specific subsets of tests.

4. Real-Time Updates:

- a. Extent Reports supports real-time updates, which means the reports can be updated dynamically as the test execution progresses.

- b. This allows you to monitor the execution status and view live results while the tests are running.

5. Customization:

- a. Extent Reports provides extensive customization options to tailor the reports according to your requirements.
- b. You can customize the report layout, styling, and add your own branding elements like logos, headers, and footers.
- c. Additionally, you can include custom charts, graphs, or additional sections to provide more insights into the test results.

Here's an example of using Extent Reports in WebDriver:

```
import com.aventstack.extentreports.ExtentReports;
import com.aventstack.extentreports.ExtentTest;
import com.aventstack.extentreports.reporter.ExtentHtmlReporter;

public class TestExecutionReport {
    public static void main(String[] args) {
        // Create an instance of ExtentReports
        ExtentReports extent = new ExtentReports();

        // Create an HTML reporter
        ExtentHtmlReporter htmlReporter = new
ExtentHtmlReporter("path/to/report.html");

        // Attach the HTML reporter to the extent instance
        extent.attachReporter(htmlReporter);

        // Create a test case
        ExtentTest test = extent.createTest("Test Case 1", "Verify Login
Functionality");
```

- In this example, we create an instance of ExtentReports and an HTML reporter (ExtentHtmlReporter).
- We attach the HTML reporter to the extent instance to generate the report in HTML format.
- We then create a test case using extent.createTest().
- Inside the test case, we add test steps using test.pass() and test.fail().
- We can also log additional details, screenshots, or attachments related to each test step.
- Finally, we mark the test as failed using test.fail() and provide the failure reason.
- After adding all the test cases and test steps, we generate the report using extent.flush().

- Extent Reports offers many more features and customization options, allowing you to create comprehensive and visually appealing reports for your WebDriver tests.

XII. Benefits of Extent Reports

The benefits of using Extent Reports in WebDriver test automation are as follows:

1. Rich and Interactive Reports:

- a. Extent Reports provides visually appealing and interactive reports that make it easy to analyze test execution results.
- b. The reports include detailed information about test cases, test steps, and their status (pass/fail). This helps in identifying issues quickly and making informed decisions.

2. Seamless Integration:

- a. Extent Reports seamlessly integrates with popular test frameworks like TestNG and JUnit.
- b. It also supports integration with build tools like Maven and Gradle, making it easy to incorporate into existing test automation projects.

3. Customization Options:

- a. Extent Reports offers extensive customization options to tailor the reports according to your requirements.

- b. You can customize the report layout, styling, and add your own branding elements like logos, headers, and footers.
- c. This allows you to create reports that align with your organization's branding guidelines.

4. Real-Time Updates:

- a. Extent Reports supports real-time updates, which means the reports can be updated dynamically as the test execution progresses.
- b. This allows you to monitor the execution status and view live results while the tests are running.

5. Dashboard View:

- a. Extent Reports provides a dashboard view that presents a summary of test execution results in the form of charts and graphs.
- b. The dashboard view gives a quick overview of key metrics like test status, pass/fail ratios, and test coverage.
- c. This helps stakeholders to get an overall understanding of the test execution outcome.

6. Historical Reports:

- a. Extent Reports allows you to generate historical reports by archiving previous test execution results.
- b. This enables you to compare and track the progress of your test automation efforts over time.

7. Easy Sharing and Collaboration:

- a. Extent Reports generates reports in HTML format, which can be easily shared with team members, stakeholders, or clients.
- b. The reports are self-contained and can be viewed using any modern web browser.

8. Extensive Test Analysis:

- a. Extent Reports provides various built-in features for in-depth test analysis, such as filtering test results based on various criteria, searching for specific test cases or keywords, and generating custom reports based on specific requirements.

Example:

```
import com.aventstack.extentreports.ExtentReports;
import com.aventstack.extentreports.ExtentTest;
import
com.aventstack.extentreports.reporter.ExtentHtmlReporter;

public class TestExecutionReport {
    public static void main(String[] args) {
        // Create an instance of ExtentReports
        ExtentReports extent = new ExtentReports();

        // Create an HTML reporter
        ExtentHtmlReporter htmlReporter = new
        ExtentHtmlReporter("path/to/report.html");

        // Attach the HTML reporter to the extent instance
        extent.attachReporter(htmlReporter);

        // Create a test case
        ExtentTest test = extent.createTest("Test Case 1", "Verify Login
        Functionality");
```

- In this example, we create an instance of ExtentReports and an HTML reporter (ExtentHtmlReporter).
- We attach the HTML reporter to the extent instance to generate the report in HTML format.
- We then create a test case using extent.createTest().
- Inside the test case, we add test steps using test.pass() and test.fail().
- We can also log additional details, screenshots, or attachments related to each test step.
- Finally, we generate the report using extent.flush().
- This example showcases the benefits of Extent Reports, such as creating visually appealing reports, adding test steps with pass/fail status, and generating the final report for analysis and sharing.

XIII. Extent reports and ExtentTest Class

- Extent Reports is a popular reporting framework in WebDriver that provides rich and interactive reports for test execution results.
- It offers features like customizable report layout, real-time updates, dashboard view, and seamless integration with test frameworks.
- The central component of Extent Reports is the ExtentTest class.
- It represents a single test case or a logical unit of test execution.

Here is an explanation of the ExtentTest class and its usage in WebDriver:

1. Creation of ExtentTest Objects:

- a. To start using ExtentTest, you need to create an instance of it.
- b. You can create multiple instances of ExtentTest, each representing a separate test case. For example:

```
ExtentTest test = extent.createTest("Test Case 1", "Verify Login Functionality");
```

- In this example, we create an ExtentTest object named "test" with the name and description provided.

2. Adding Test Steps:

- a. Once the ExtentTest object is created, you can add test steps using various methods like pass(), fail(), skip(), etc.
- b. These methods represent the outcome of each test step.

For example:

```
test.pass("Step 1: Launch application");  
test.pass("Step 2: Enter username");  
test.pass("Step 3: Enter password");  
test.fail("Step 4: Click on login button");
```

- Here, we use the pass() method for successful steps and the fail() method for steps that encounter failures.
- You can also use the skip() method to mark steps that are skipped.

3. Logging Details:

- a. You can log additional details, screenshots, or attachments related to each test step using the log() method.

For example:

```
test.log(Status.INFO, "Additional details about the test step");  
test.addScreenCaptureFromPath("path/to/screenshot.png");  
test.addAttachment("path/to/attachment.txt", "Attachment  
Description");
```

- The `log()` method allows you to provide a status for the log message (e.g., INFO, WARNING, ERROR) and a description of the log message.

4. Marking Test Status:

- a. After executing all the test steps, you can mark the overall status of the test case using methods like `pass()`, `fail()`, or `skip()`.

For example:

```
test.pass("Test case passed successfully");
test.fail("Test case failed due to an assertion error");
test.skip("Test case skipped due to a known issue");
```

- These methods allow you to provide an optional message explaining the outcome of the test case.

5. Nested Test Steps:

- a. ExtentTest supports nested test steps, which can be useful for hierarchical representation of test execution.
- b. You can create child test steps using the `createNode()` method.

For example:

```
ExtentTest step1 = test.createNode("Step 1");
step1.pass("Sub-step 1.1: Perform some action");
step1.pass("Sub-step 1.2: Verify the result");

ExtentTest step2 = test.createNode("Step 2");
step2.pass("Sub-step 2.1: Perform another action");
```

- This allows you to organize the test steps in a structured manner.
- Overall, the ExtentTest class provides a convenient way to capture and represent the test execution details in Extent Reports.
- It allows you to create meaningful and informative reports for your WebDriver tests.

XIV. Methods of ExtentReports and ExtentTest Classes (startTest, endTest, Log, Flush)

- The ExtentReports and ExtentTest classes in WebDriver offer various methods to manage and customize the reports.

Here's an explanation of some commonly used methods:

1. ExtentReports Class:

- startTest(String testName, String description):
 - Starts a new test and returns an ExtentTest object.
 - You can provide the name and description of the test as parameters.

Example:

```
ExtentTest test = extent.startTest("Test Name", "Test Description");
```

- `endTest(ExtentTest test)`:
 - Ends the specified test.
 - It is recommended to call this method at the end of each test case to mark its completion.

Example:

```
extent.endTest(test);
```

- `flush()`:
 - Flushes the test information to the report.
 - It is typically called after all the tests have completed execution to generate the final report.

Example:

```
extent.flush();
```

2. ExtentTest Class:

- `log(LogStatus status, String details)`:
 - Logs the status and details of a particular test step.
 - You can specify the status (e.g., PASS, FAIL, INFO) and provide a description of the test step.

Example:

```
test.log(LogStatus.PASS, "Step 1: Click on Login button");
```

- `log(LogStatus status, String details, String screenshotPath)`:
 - Logs the status, details, and attaches a screenshot to the test step.

Example:

```
test.log(LogStatus.FAIL, "Step 2: Verify error message",  
"path/to/screenshot.png");
```

`throwable)`:

- Logs the status, details, and an exception associated with the test step.

Example:

```
test.log(LogStatus.ERROR, "Step 3: Handle exception", exception);
```

- `log(LogStatus status, Markup markup)`:
 - Logs the status and details using HTML markup.

Example:

```
test.log(LogStatus.INFO, MarkupHelper.createCodeBlock("Code  
snippet"));
```

- These methods provide flexibility in managing and customizing the test reports generated by ExtentReports and ExtentTest classes in WebDriver.
- You can use them to create detailed and visually appealing reports for your test automation projects.

XV. Capturing screenshots in Extent Reports- getScreenShotsAs()

- In Extent Reports, you can capture screenshots of your test execution and include them in your report using the getScreenShotsAs() method.

Here's an explanation of how to use this method in WebDriver:

1. Capturing a Screenshot:

- a. To capture a screenshot in WebDriver, you can use the TakesScreenshot interface.
- b. The getScreenShotsAs() method in Extent Reports requires an OutputType parameter to specify the format of the screenshot (e.g., FILE, BYTES).

Here's an example of capturing a screenshot as a file:

```
// Capture a screenshot as a File
File screenshot = ((TakesScreenshot)
driver).getScreenshotAs(OutputType.FILE);
```

2. Adding the Screenshot to Extent Report:

- a. Once you have captured the screenshot, you can add it to your Extent Report using the `test.log()` method with the appropriate log status and details.

Here's an example:

```
// Capture screenshot and add it to the Extent Report
test.log(LogStatus.INFO, "Step 1: Click on Login button",
test.addScreenCapture(screenshot.getAbsolutePath()));
```

- In the above example, the `addScreenCapture()` method is used to attach the screenshot to the report.
- You can provide the absolute path of the screenshot file as a parameter.
- By using the `getScreenShotsAs()` method and `addScreenCapture()` method together, you can capture screenshots during your test execution and include them in your Extent Report.
- This helps in providing visual evidence of the test steps and any failures encountered.

8. Automation Frameworks-I

Information

I. Introduction to Selenium Frameworks

- Selenium is a powerful and popular automation testing tool for web applications.

- To enhance the capabilities of Selenium and make the automation process more efficient, various frameworks have been developed.
- These frameworks provide a structured approach to writing and managing Selenium test scripts, making it easier to maintain, scale, and collaborate on automation projects.

Here's an introduction to some popular Selenium frameworks:

1. TestNG:

- a. TestNG is a widely used testing framework that offers advanced features for test management, parallel execution, test configuration, and reporting.
- b. It provides annotations for defining test methods, supports data-driven testing, and allows test execution in parallel.
- c. TestNG integrates well with Selenium WebDriver and is compatible with various build tools.

2. JUnit:

- a. JUnit is a popular unit testing framework for Java applications.
- b. It provides annotations and assertions to define and verify test cases.
- c. Although primarily designed for unit testing, JUnit can be used effectively with Selenium for web application testing.
- d. JUnit offers features like test suites, parameterized tests, and test runners.

3. Cucumber:

- a. Cucumber is a behavior-driven development (BDD) framework that enables collaboration between business stakeholders, developers, and testers.
- b. It uses a plain-text format called Gherkin to describe the application's behavior in terms of scenarios and steps.
- c. Cucumber allows test scenarios to be written in a human-readable format and provides the ability to automate these scenarios using step definitions.

4. Selenium Grid:

- a. Selenium Grid is a distributed testing framework that enables running tests on multiple machines or browsers in parallel.
- b. It allows for cross-browser and cross-platform testing, making it easier to ensure the compatibility and responsiveness of web applications across different environments.
- c. Selenium Grid helps in achieving faster test execution by distributing the tests across multiple machines.

5. Page Object Model (POM):

- a. POM is a design pattern that promotes the creation of reusable and maintainable test code.
- b. It separates the test logic from the UI elements of a web page, making the test code more readable and maintainable.

- c. POM provides a structured approach to defining and interacting with web page elements, improving the overall test structure and reducing code duplication.
- These frameworks offer different features and benefits, allowing you to choose the one that best fits your project requirements and team preferences.
- They provide a systematic way of organizing and managing your Selenium test automation efforts, making it easier to write robust and scalable test scripts.

II. Types of Selenium Frameworks

- There are several types of Selenium frameworks that are commonly used in test automation.
- Each framework has its own unique features and benefits, catering to different needs and preferences.

Here are some popular types of Selenium frameworks:

1. Data-driven Framework:

- a. The data-driven framework focuses on separating the test data from the test scripts.
- b. It allows for testing different sets of data without modifying the test logic.
- c. Test data is stored in external sources like Excel or CSV files, and the framework retrieves the data dynamically during test execution.

2. Keyword-driven Framework:

- a. The keyword-driven framework involves creating a set of keywords or actions that represent different test steps or operations.
- b. Test scripts are written using these keywords, making the test scenarios more readable and understandable.
- c. The framework maps the keywords to the actual test code implementation.

3. Hybrid Framework:

- a. The hybrid framework is a combination of different frameworks, such as data-driven and keyword-driven frameworks.
- b. It provides the flexibility to leverage the advantages of multiple frameworks based on the specific requirements of the application and the testing needs.

- These are some of the commonly used types of Selenium frameworks.
- The choice of framework depends on the project requirements, the complexity of the application, the team's expertise, and the desired level of code reusability and maintainability.
- It's important to evaluate the pros and cons of each framework and choose the one that best suits your needs

III. Introduction to Data-Driven Test Framework

- A data-driven test framework is a test automation framework that separates the test data from the test scripts.
- It allows for executing the same test logic with different sets of data, making test scenarios more flexible and reusable.
- The data can be stored in external sources like Excel sheets, CSV files, databases, or even JSON/XML files.
- In a data-driven test framework, the test scripts are designed to read the test data from the external sources and use it during test execution.
- The framework provides the necessary functionality to retrieve and manage the test data, ensuring that the tests are executed with the desired data combinations.

1. Benefits of a Data-Driven Test Framework:

- a. Reusability: Test scripts can be reused with different sets of data, reducing the effort required to create and maintain tests.

2. Scalability:

- a. The framework can handle a large volume of test data, allowing for comprehensive test coverage.

3. Flexibility:

- a. Test scenarios can be easily modified by updating the test data, without making changes to the test logic.

4. Maintenance:

- a. Changes to the test data can be made independently of the test scripts, simplifying maintenance.

5. Test Coverage:

- a. Different combinations of test data can be tested, including edge cases and boundary values.

6. Separation of Concerns:

- a. Test data and test logic are kept separate, promoting better code organization.

- Example of a Data-Driven Test Framework:

- Let's consider a scenario where you have a login functionality that needs to be tested with different sets of credentials.
- Instead of hard-coding the usernames and passwords within the test script, you can store them in an external data source like an Excel sheet.
- The test script can then read the data from the Excel sheet and execute the login test case with each set of credentials.

- This approach allows you to easily add or modify test data without changing the test script.
- You can have multiple test cases using different sets of data, making it convenient to test various scenarios and validate the login functionality with different credentials.
- Overall, a data-driven test framework provides flexibility, reusability, and scalability in test

automation by decoupling the test data from the test scripts.

- It helps in achieving comprehensive test coverage and simplifies maintenance efforts.
- Overall, while data-driven test frameworks offer significant advantages in terms of reusability, scalability, and flexibility, they may require additional effort and expertise during setup and maintenance.
- Organizations should carefully evaluate their testing needs and resources before deciding to adopt a data-driven approach.

IV. Apache POI

- Apache POI is an open-source Java library that provides APIs for reading and writing data in Microsoft Office formats such as Excel, Word, and PowerPoint.
- It allows Java developers to create, modify, and extract data from Office documents programmatically.

Key features and functionalities of Apache POI include:

1. Excel Manipulation:

- a. Apache POI provides comprehensive support for working with Excel files.
- b. It allows you to create new Excel workbooks, read existing ones, modify cell values, apply formatting, add formulas, and perform various operations on worksheets and rows.

2. Word Processing:

- a. Apache POI includes APIs for working with Word documents.
- b. It enables you to create and modify Word files, add text, tables, images, and formatting, manipulate document properties, and perform other operations related to Word processing.

3. PowerPoint Presentations:

- a. Apache POI allows you to create, modify, and extract data from PowerPoint presentations.
- b. You can add slides, shapes, text, images, and apply formatting to create dynamic presentations.

4. Reading and Writing Data:

- a. Apache POI provides APIs for reading and writing data in Office file formats.
- b. It supports different data types, formulas, styles, and formatting options.

5. Cross-platform Compatibility:

- a. Apache POI is platform-independent and can be used on different operating systems such as Windows, macOS, and Linux.

Here's an example of how to use Apache POI to read data from an Excel file:

```
import org.apache.poi.ss.usermodel.*;
import org.apache.poi.xssf.usermodel.XSSFWorkbook;
import java.io.File;
import java.io.FileInputStream;
import java.io.IOException;
```

```
for (Row row : sheet) {
    for (Cell cell : row) {
        CellType cellType = cell.getCellType();
        if (cellType == CellType.STRING) {
            System.out.print(cell.getStringCellValue() + "\t");
        } else if (cellType == CellType.NUMERIC) {
            System.out.print(cell.getNumericCellValue() + "\t");
        } else if (cellType == CellType.BOOLEAN) {
            System.out.print(cell.getBooleanCellValue() + "\t");
        }
    }
    System.out.println();
}

workbook.close();
file.close();
} catch (IOException e) {
    e.printStackTrace();
}
}
```

- This example demonstrates how to read data from an Excel file using Apache POI's XSSF implementation for the newer .xlsx file format.
- The code opens the file, accesses the first sheet, and iterates over each row and cell, printing the cell values.
- Apache POI is a powerful library for working with Office documents in Java, providing a wide range of features for data manipulation and extraction.

9. Automation Frameworks-II

Information

I. Introduction to Keyword-Driven Test Framework

- Keyword-driven testing is a test automation framework that allows testers to design and execute tests using a set of keywords or action words.
- In this framework, the test cases are separated from the test logic by defining keywords that represent specific test actions or operations.
- The basic idea behind a keyword-driven framework is to create a table or spreadsheet that maps keywords to specific test steps or actions.
- Each test case is defined by a sequence of keywords, and the framework executes the corresponding actions based on the keywords.

The advantages of using a keyword-driven framework include:

1. **Modularity:** The test cases are modular and can be easily modified or reorganized by rearranging the keywords.
 2. **Reusability:** Keywords can be reused across multiple test cases, reducing duplication of effort.
 3. **Abstraction:** The test cases are written in a language that is closer to the business domain, making them easier to understand for non-technical stakeholders.
 4. **Maintainability:** Changes to the underlying implementation can be isolated within the framework, minimizing the impact on test cases.
- However, it's important to note that keyword-driven frameworks require careful planning and design.
 - Defining appropriate keywords and maintaining the mapping between keywords and test actions is crucial for the success of this approach.
 - Implementing a keyword-driven framework involves developing the framework itself, which includes defining the keywords, implementing the actions associated with each keyword, and providing mechanisms for executing the test cases using the keywords.
 - There are several tools and frameworks available that support keyword-driven testing, such as Robot Framework, Cucumber, and FitNesse.

- These tools provide additional features and capabilities to simplify the creation and execution of keyword-driven tests.
- Overall, the keyword-driven test framework provides a structured and flexible approach to test automation, allowing testers to create tests that are easy to understand, maintain, and execute.

II. Uses of Keyword-Driven Test Framework

- The keyword-driven test framework offers several benefits and can be utilized in various scenarios.

Some of the common uses of the keyword-driven test framework are:

1. Test Automation:

- a. The keyword-driven framework is primarily used for test automation.
- b. It provides a structured approach to define and execute test cases using keywords, making it easier to create automated tests without extensive programming knowledge.

2. Cross-Browser Testing:

- a. With the keyword-driven framework, you can define keywords for actions like opening a browser, navigating to a URL, interacting with web elements, and more.
- b. This makes it suitable for cross-browser testing, as you can reuse the same keywords across

different browsers without modifying the test cases.

3. Data-Driven Testing:

- a. The keyword-driven framework can be combined with data-driven testing techniques.
- b. You can define keywords for test actions and use test data from external sources like spreadsheets or databases.
- c. This allows you to execute the same test steps with different input values, making it useful for testing various scenarios and data combinations.

4. Test Maintenance and Reusability:

- a. The keyword-driven approach promotes modularity and reusability.
- b. Test cases are defined using keywords, which can be easily modified or rearranged.
- c. When there are changes in the application under test, you can update the keywords or test steps in the framework, and the changes will be reflected in all the test cases using those keywords.
- d. This simplifies test maintenance and reduces the effort required to update test cases.

5. Collaboration and Non-Technical Involvement:

- a. The keyword-driven framework enables collaboration between technical and non-technical stakeholders.
- b. Test cases are written using keywords that are closer to the business domain, making them

easier to understand for non-technical team members.

- c. This allows domain experts or business analysts to actively participate in the test case creation and review process.

6. Test Reporting and Traceability:

- a. The keyword-driven framework can generate comprehensive test reports that provide insights into test execution status, test coverage, and failures.

These reports can help track the progress of testing activities and provide visibility into the overall quality of the application.

- b. Additionally, the mapping between keywords and test actions allows for traceability, making it easier to identify which keywords are associated with specific test steps.

- Overall, the keyword-driven test framework provides a flexible and structured approach to test automation, offering benefits such as maintainability, reusability, collaboration, and efficient test execution.
- Its uses can extend to various testing scenarios, making it a valuable framework in the software testing process.

V. Introduction to Hybrid Driven Test Framework

- The hybrid-driven test framework is a combination of data-driven and keyword-driven frameworks.

- It leverages the strengths of both frameworks to create a flexible and scalable testing solution.
- In a hybrid-driven framework, test cases are designed using keywords, and the test data is stored in external data sources such as spreadsheets or databases.

Here is an overview of the hybrid-driven test framework:

1. Keyword Repository:
 - a. Create a repository that maps keywords to the corresponding test steps or actions, similar to the keyword-driven framework.
2. Test Data Source:
 - a. Use external data sources such as spreadsheets or databases to store the test data.
 - b. Each test case can have associated test data in the data source.
3. Test Case Design:
 - a. Design test cases using keywords from the repository, just like in the keyword-driven framework.
 - b. Test steps are defined using keywords, and the corresponding test data is fetched from the data source.
4. Test Execution Engine:
 - a. Develop an execution engine that reads the test cases, retrieves keywords and test data, and executes the test steps accordingly.

- b. The engine should have the capability to handle both keywords and test data.

5. Modularity and Reusability:

- a. The hybrid-driven framework promotes modularity and reusability by allowing the reuse of keywords and test data across multiple test cases.
- b. This helps in reducing duplication and improving maintenance.

6. Flexibility and Scalability:

- a. The hybrid-driven framework provides flexibility to handle complex test scenarios by combining keyword-driven and data-driven approaches.
- b. It allows for easy customization and scalability as the testing needs evolve.

7. Reporting and Logging:

- a. Generate comprehensive reports that provide insights into the test execution, including test case status, failures, and other relevant details.
 - b. Implement logging mechanisms to capture important information during test execution.
- The hybrid-driven framework offers the advantages of both data-driven and keyword-driven frameworks, allowing testers to design flexible and maintainable test cases while leveraging external data sources for test data management.

- It provides a balance between reusability, scalability, and customization, making it suitable for various testing scenarios.

10. Page Object Model

Information

I. Why Page Object Model?

- The Page Object Model (POM) is a design pattern widely used in Selenium WebDriver automation testing.
- It aims to enhance the maintainability, reusability, and readability of test automation code by providing a structured and modular approach to interact with web pages.
- The Page Object Model (POM) is a design pattern used in Selenium WebDriver automation testing.
- It is a way to structure and organize the automation code to improve the maintainability, reusability, and readability of tests.
- In the Page Object Model, each web page in the application under test is represented as a separate class called a "Page Object."
- The Page Object encapsulates the elements and functionality of the web page, providing methods to

interact with those elements and perform actions on them.

- The main idea behind the Page Object Model is to separate the test code from the underlying implementation details of the web pages.
- This separation allows for better maintainability and reduces the impact of changes in the application's UI.
- Test cases interact with the web elements through the Page Objects, using the methods defined in the Page Objects rather than directly accessing the web elements in the tests.

Benefits of using the Page Object Model in WebDriver automation testing:

1. Reusability:

- a. Page Objects can be reused across multiple test cases, avoiding duplication of code and promoting code reusability.

2. Maintainability:

- a. With the POM, if there are changes in the UI or functionality of a web page, you only need to update the corresponding Page Object, rather than modifying multiple test cases.

3. Readability:

- a. Page Objects provide a higher level of abstraction, making the test code more readable and easier to understand.

4. Scalability:

- a. The POM allows for easy scalability as the number of test cases and web pages grows.
- b. New Page Objects can be added to represent new pages, and existing Page Objects can be modified or extended as needed.

5. Collaboration:

- a. The separation of concerns in the POM allows for better collaboration between developers and testers.
 - b. Developers can focus on building and maintaining the Page Objects, while testers can write tests using the provided methods and interact with the web elements without worrying about the implementation details.
- Overall, the Page Object Model is a popular approach in WebDriver automation testing as it promotes best practices, improves test code organization, and enhances test maintainability and reusability.

II. What is Page Factory?

- Page Factory is a concept introduced in Selenium WebDriver to simplify and enhance the implementation of the Page Object Model (POM) design pattern.
- It provides a convenient way to initialize and locate web elements in the Page Objects.
- The Page Factory class in Selenium WebDriver provides annotations that can be used to identify and initialize web elements on a web page.

- By using these annotations, the initialization of web elements is automated, reducing the need for repetitive code.

To use Page Factory, the following steps need to be followed:

1. Import the required classes:
 - a. Import the necessary classes from the `org.openqa.selenium.support` package, such as `PageFactory` and the annotations like `FindBy`, `FindByBys`, `CacheLookup`, etc.
2. Initialize the Page Factory:
 - a. In the constructor of the Page Object class, use the `PageFactory.initElements()` method to initialize the elements on the page.
 - b. Pass the `WebDriver` instance and the current Page Object (usually `this`) as parameters.
3. Use annotations to locate elements:
 - a. Annotate the web elements on the page with the appropriate annotations provided by Page Factory.
 - b. For example, use `@FindBy` to locate a single element, `@FindByBys` for locating a group of elements with a set of locators, and `@CacheLookup` to cache the element once it is found.

- By using Page Factory, you can avoid writing explicit code for finding and initializing web elements.
- Instead, the elements are automatically initialized when the Page Object is created, and you can directly interact with the elements in your test methods.

III. Advantages of POM Webdriver

- The Page Object Model (POM) is a popular design pattern used in Selenium WebDriver for building scalable and maintainable test automation frameworks.

Advantages:

1. Improved code organization:

- a. POM promotes the separation of concerns by separating the test logic from the page structure and interactions.
- b. This makes the code more organized, readable, and maintainable.

2. Reusability of code:

- a. With POM, page objects encapsulate the behaviour and interactions of web pages.
- b. These page objects can be reused across multiple test cases, reducing code duplication and improving efficiency.

3. Easy maintenance:

- a. When the structure or elements of a web page change, you only need to update the corresponding page object, rather than modifying multiple test cases.
- b. This makes maintenance easier and minimizes the impact of changes.

4. Enhanced collaboration:

- a. POM facilitates collaboration among team members working on different layers of the automation framework.
- b. Testers can focus on writing test cases using the page objects, while developers can work on updating the page objects based on changes in the application.

5. Improved test readability:

- a. By using meaningful method and variable names in the page objects, test cases become more descriptive and readable, making it easier to understand the purpose and flow of each test.

6. Easy troubleshooting:

- a. With POM, if a test case fails, you can easily locate and debug the issue within the corresponding page object, as it contains all the interactions with the web page.
- b. This saves time and effort in troubleshooting failures.

7. Scalability:

- a. POM allows for easy scalability as new page objects can be added for new web pages or features.
- b. Existing test cases can be extended by reusing the existing page objects.

8. Parallel execution:

- a. POM provides a solid foundation for parallel test execution by ensuring that each test case operates independently on its respective page object.
 - b. This enables faster execution and efficient use of testing resources.
- In summary, POM brings numerous advantages to Selenium WebDriver test automation, including code organization, reusability, easy maintenance, collaboration, readability, troubleshooting, scalability, and support for parallel execution.
 - It promotes best practices and helps build robust and maintainable test automation frameworks.

IV. Implementing POM using Selenium WebDriver

Here's an example of implementing POM using Selenium WebDriver in Java:

```
// LoginPage.java - Page Object Class
public class LoginPage {
    @FindBy(id = "username")
    private WebElement usernameInput;

    @FindBy(id = "password")
    private WebElement passwordInput;

    @FindBy(id = "loginButton")
```

```
public void clickLoginButton() {
    loginButton.click();
}

// TestClass.java - Test Class
public class TestClass {
    private WebDriver driver;
    private LoginPage loginPage;

    @BeforeClass
    public void setUp() {
        // Set up WebDriver and initialize page objects
        driver = new ChromeDriver();
        loginPage = PageFactory.initElements(driver, LoginPage.class);
    }

    @Test
    public void loginTest() {
        // Test case using page objects
        driver.get("https://example.com/login");

        loginPage.enterUsername("username");
        loginPage.enterPassword("password");
        loginPage.clickLoginButton();
    }
}
```

- In this example, the LoginPage class represents the login page with its web elements and methods to interact with them.
- The TestClass uses the page object to perform the login action in the test case.
- By following this approach, you can create more page objects for different web pages and reuse them across multiple test cases, leading to a scalable and maintainable test automation framework using POM.

V. Implementing POM with Page Factory

- Implementing the Page Object Model (POM) with Page Factory in Selenium WebDriver involves using

the PageFactory class provided by the Page Factory pattern to initialize the elements within a page object.

Here's an example of how to implement POM with Page Factory in Java:

1. Create the Page Object class:

```
public class LoginPage {  
    @FindBy(id = "username")  
    private WebElement usernameInput;  
  
    @FindBy(id = "password")  
    private WebElement passwordInput;  
  
    @FindBy(id = "loginButton")  
    private WebElement loginButton;  
  
    public void enterUsername(String username) {  
        usernameInput.sendKeys(username);  
    }  
  
    public void enterPassword(String password) {  
        passwordInput.sendKeys(password);  
    }  
  
    public void clickLoginButton() {  
        loginButton.click();  
    }  
}
```

2. Create the Test class:

```
public class TestClass {  
    private WebDriver driver;  
    private LoginPage loginPage;  
  
    @BeforeClass
```

```
@AfterClass
public void tearDown() {
    // Clean up resources
    driver.quit();
}
```

- In this example, the LoginPage class represents the login page with its web elements defined using the @FindBy annotation.
- The constructor of the LoginPage class uses PageFactory.initElements(driver, this) to initialize the elements using the provided WebDriver instance.
- The TestClass uses the LoginPage object to perform the login action in the test case.

- Using Page Factory simplifies the initialization of elements within the page object, eliminating the need for repetitive `driver.findElement` calls.
- It enhances code readability and maintainability.
- Additionally, it allows for lazy initialization of elements, improving performance by only initializing elements when they are accessed or interacted with.
- By implementing POM with Page Factory, you can create scalable and maintainable test automation frameworks in Selenium WebDriver.

VI. Implementing Hybrid Test Framework

- Implementing a Hybrid Test Framework using Selenium WebDriver involves combining multiple testing approaches, such as Data-Driven Testing, Keyword-Driven Testing, and Behavior-Driven Testing, to create a flexible and robust framework.

Here's an overview of how you can implement a Hybrid Test Framework:

1. Identify Test Scenarios:
 - a. Identify the test scenarios for your application.
 - i. Divide the scenarios into logical groups based on their functionality.
2. Design Test Data:
 - a. Define the test data required for each test scenario.

- b. Organize the test data in a structured format, such as Excel sheets or CSV files.
3. Create Page Objects:
 - a. Implement the Page Object Model (POM) to represent the web pages of your application.
 - b. Create separate classes for each page and define the web elements and corresponding methods.
4. Implement Test Scripts:
 - a. Write test scripts using a combination of Data-Driven and Keyword-Driven approaches.
 - b. Use the Page Objects to interact with the web elements and perform actions on the pages.
 - c. Retrieve test data from the data sources and pass it to the test scripts.
5. Implement Test Configuration:
 - a. Define a configuration file or class to manage test environment settings, such as browser type, URL, timeouts, etc.
 - b. Use properties files or configuration files to store and retrieve these settings.
6. Integrate Reporting and Logging:
 - a. Integrate a reporting and logging mechanism to capture test execution results and generate reports.
 - b. Use tools like Extent Reports, TestNG reporting, or custom logging libraries for detailed test reporting.

7. Implement Error Handling and Exception Management:

- a. Implement error handling and exception management techniques to handle unexpected exceptions and failures during test execution.
- b. Use try-catch blocks, assertions, and appropriate error handling mechanisms.

8. Implement Test Configuration and Management:

- a. Implement test configuration management to handle different environments (e.g., Dev, QA, Staging, Production).
- b. Maintain separate configuration files or properties for each environment.

9. Implement Test Execution Engine:

- a. Create a test execution engine class to orchestrate the execution of test scripts.
- b. Initialize the WebDriver and other required objects.
- c. Configure the test environment and load test data.
- d. Call the test scripts and pass the required parameters.

10. Run and Monitor Tests:

- a. Execute the test scripts using a test runner or build automation tool, such as TestNG or Maven.
- b. Monitor the test execution and collect the test results.

- By implementing a Hybrid Test Framework, you can combine the strengths of different testing approaches and create a flexible and scalable framework for your Selenium WebDriver tests.
- This approach allows for efficient test design, easy test maintenance, and better test coverage.

11. BDD Framework with Cucumber and Selenium Grid

Information

I. Agile Testing

- Agile Testing is a testing approach that aligns with the principles of Agile software development methodologies.
- It emphasizes collaboration, iterative development, and early and continuous testing throughout the software development lifecycle.
- When implementing Agile Testing with Selenium

WebDriver, here are some key considerations:

1. Embrace Iterative Testing:

- a. Agile projects work in short iterations or sprints, with frequent releases and feedback loops.
- b. Testers should focus on testing small increments of functionality within each sprint.

- c. Selenium WebDriver allows testers to automate tests for each sprint and provide immediate feedback.

2. Collaborate with the Agile Team:

- a. Agile Testing promotes close collaboration between testers, developers, and other stakeholders.
- b. Testers should actively participate in Agile ceremonies like daily stand-ups, sprint planning, and retrospectives.
- c. Work closely with developers to understand user stories and acceptance criteria for effective test coverage.

3. Continuous Integration and Continuous Testing:

- a. Implement Continuous Integration (CI) practices to build, test, and integrate changes frequently.
- b. Use tools like Jenkins, Bamboo, or GitLab CI/CD to automate build and test execution.
- c. Selenium WebDriver tests can be integrated into the CI pipeline to trigger tests on every code commit.

4. Prioritize Test Cases:

- a. Due to time constraints in Agile projects, it's essential to prioritize test cases based on their criticality and business impact.
- b. Focus on testing high-priority features, critical functionalities, and areas with higher risks.
- c. Use risk-based testing approaches to identify and prioritize test cases effectively.

5. Use Test Automation:

- a. Automation is crucial in Agile Testing to ensure faster test execution and regression testing.
- b. Selenium WebDriver provides a robust automation framework for web application testing.
- c. Automate repetitive and stable test cases to achieve faster feedback and enable faster releases.

6. Agile Test Design Techniques:

- a. Use exploratory testing techniques to complement scripted test cases and uncover hidden defects.
- b. Incorporate Agile Test Design techniques like Behavior-Driven Development (BDD) or Acceptance Test-Driven Development (ATDD).
- c. Tools like Cucumber, JBehave, or SpecFlow can be integrated with Selenium WebDriver to facilitate BDD-style testing.

7. Continuous Feedback and Adaptation:

- a. Agile Testing encourages continuous feedback and adaptation based on customer and stakeholder feedback.
- b. Capture and communicate test results, defects, and improvement areas regularly.
- c. Participate in sprint retrospectives to identify areas for process improvement and refinement.

- By adopting Agile Testing principles and leveraging the capabilities of Selenium WebDriver, teams can ensure timely and high-quality software delivery in an Agile development environment.
- The close collaboration, continuous testing, and test automation provided by Selenium WebDriver support the Agile values of early and frequent feedback, adaptability, and continuous improvement.

II. Behaviour Driven Development (BDD)

- Behaviour-Driven Development (BDD) is a software development approach that emphasizes collaboration and communication between developers, testers, and business stakeholders.
- It aims to align business requirements with the software development process.
- When implementing BDD with Selenium WebDriver, here are some key points to consider:

1. Shared Understanding:

- a. BDD encourages collaboration among stakeholders to ensure a shared understanding of the desired behavior of the software.
- b. Business stakeholders, developers, and testers work together to define and refine the application's behavior using a common language.

2. User Stories and Scenarios:

- a. BDD relies on user stories to capture functional requirements from a user's perspective.
- b. User stories are typically written in a structured format called the "Given-When-Then" format, which defines the preconditions, actions, and expected outcomes of a specific scenario.

3. Feature Files:

- a. BDD scenarios are typically written in feature files using a natural language syntax, such as Gherkin.
- b. Feature files serve as a documentation and communication tool, allowing stakeholders to understand and review the expected behaviour of the software.
- c. Feature files can be stored in a version control system and serve as a living documentation throughout the project.

4. Step Definitions:

- a. Step definitions map the steps in the feature files to the corresponding automation code.
- b. Each step in a scenario is implemented as a reusable method in the step definition files.
- c. Step definitions bridge the gap between the business-readable feature files and the actual automation code using Selenium WebDriver.

5. Automation with Selenium WebDriver:

- a. Selenium WebDriver is used to automate the actions and validations specified in the BDD scenarios.

- b. Each step in a scenario is mapped to the corresponding Selenium WebDriver code to interact with the web application.
- c. WebDriver APIs are used to perform actions like clicking buttons, filling forms, navigating pages, and verifying expected outcomes.

6. Integration with BDD Frameworks:

- a. BDD frameworks like Cucumber, JBehave, or SpecFlow can be used to implement BDD with Selenium WebDriver.
- b. These frameworks provide tools and libraries to parse the feature files, execute the scenarios, and generate reports.
- c. They also offer integration with Selenium WebDriver, allowing seamless execution of the automated tests.

7. Collaboration and Feedback:

- a. BDD encourages continuous collaboration between developers, testers, and business stakeholders throughout the development process.
- b. Stakeholders can review and provide feedback on the feature files, ensuring that the expected behavior is accurately captured.
- c. BDD scenarios serve as a common reference point for discussions and clarifications, promoting better communication and shared understanding.

- By adopting BDD principles and combining them with the power of Selenium WebDriver, teams can enhance their collaboration, improve requirements clarity, and develop high-quality software that meets business needs.
- BDD helps align technical implementation with business expectations, and Selenium WebDriver enables the automation of BDD scenarios for efficient and reliable testing.

III. Advantages and Limitations of BDD

Advantages of BDD (Behavior-Driven Development) with WebDriver:

1. Improved Collaboration:
 - a. BDD promotes collaboration among developers, testers, and business stakeholders.
 - b. It encourages open communication and a shared understanding of the desired behaviour of the software.
2. Clear Requirements:
 - a. BDD uses a common language and a structured format (Given-When-Then) to define requirements.
 - b. This makes it easier to capture and understand the expected behavior of the software.
3. Business-Focused Testing:

- a. BDD focuses on the business value of the software.
- b. It helps ensure that the software meets the business requirements and delivers value to the end users.

4. Enhanced Test Coverage:

- a. BDD encourages defining scenarios from a user's perspective, which helps identify various test scenarios and ensures comprehensive test coverage.

5. Early Feedback:

- a. BDD allows stakeholders to review and provide feedback on the feature files and scenarios early in the development process.
- b. This helps catch issues and misunderstandings early, leading to faster resolution and improved software quality.

6. Automation Readiness:

- a. BDD scenarios are written in a structured format that can be easily automated.
- b. By using WebDriver, the scenarios can be automated efficiently, allowing for reliable and repeatable testing.

7. Living Documentation:

- a. BDD feature files serve as living documentation that describes the behaviour of the software.
- b. They can be used as a reference point for developers, testers, and business stakeholders,

making it easier to maintain and update the documentation as the software evolves.

8. Limitations of BDD with WebDriver:

- a. Learning Curve: BDD may have a learning curve for teams who are new to the concept and the associated tools.
- b. It requires understanding the syntax and best practices of writing feature files and implementing step definitions.

9. Overhead:

- a. BDD introduces an additional layer of abstraction between the business requirements and the automation code.
- b. This can result in some overhead in terms of maintaining and updating the feature files and step definitions.

10. Initial Setup:

- a. Setting up the BDD framework, integrating it with WebDriver, and configuring the environment may require some initial effort and time investment.

11. Maintenance Effort:

- a. As the software evolves, the feature files and step definitions may need to be updated to reflect the changes in the application.

- b. This requires ongoing maintenance effort to ensure the scenarios remain accurate and relevant.

12. Limited Scope:

- a. BDD is primarily focused on functional testing and may not cover all types of testing, such as performance or security testing.
 - b. Additional frameworks or tools may be required to address these testing aspects.
- Despite these limitations, BDD with WebDriver offers numerous advantages in terms of collaboration, clarity of requirements, business-focused testing, and automation readiness.
 - It helps teams align their development efforts with business expectations and delivers high-quality software that meets user needs.

IV. Cucumber Fundamentals

- Cucumber is a tool that supports Behavior-Driven Development (BDD) by enabling teams to define and execute executable specifications in a human-readable format.
- It allows collaboration among developers, testers, and business stakeholders by using a common language to describe the behavior of the software.

1. Fundamentals of Cucumber with WebDriver:

- a. Feature Files: Cucumber uses feature files to define the desired behavior of the software.
- b. Feature files are written in a plain text format and use the Gherkin language, which is a business-readable domain-specific language.
- c. Feature files contain scenarios that describe various test cases or user stories.

2. Gherkin Syntax:

- a. Gherkin is a structured language used to write feature files.
- b. It follows a Given-When-Then format, also known as the "GWT" or "BDD" syntax.
- c. The Given step describes the initial state or preconditions, the When step describes the actions performed, and the Then step describes the expected outcomes.

3. Step Definitions:

- a. Step definitions map the steps mentioned in the feature files to the corresponding automation code.
- b. Step definitions are implemented using programming languages such as Java, and they define the actions to be performed for each step.

4. Scenario Outline:

- a. Cucumber provides a Scenario Outline feature that allows defining a template for scenarios with varying input values.

- b. It uses placeholders, known as data tables, to provide different input values for each scenario iteration.

5. Tags:

- a. Cucumber allows the use of tags to organize and filter scenarios.
- b. Tags can be used to group related scenarios or to specify certain scenarios to be included or excluded during test execution.

6. Data Tables:

- a. Cucumber supports the use of data tables in feature files.
- b. Data tables allow defining tabular data that can be used as input or output for scenarios.
- c. They are useful for representing test data or expected results in a structured format.

7. Hooks:

- a. Cucumber provides hooks that allow executing certain code before or after specific events, such as scenario execution or test suite initialization.
- b. Hooks can be used for setup and teardown activities or for capturing screenshots or logs.

8. Reporting:

- a. Cucumber generates detailed reports that provide insights into the test execution results.
- b. These reports can include information such as the number of scenarios passed, failed, or

pending, as well as execution times and error details.

- By combining Cucumber with WebDriver, teams can create human-readable and executable specifications, automate the scenarios using WebDriver, and generate reports that provide visibility into the test execution results.
- This helps in improving collaboration, ensuring clarity of requirements, and delivering high-quality software that meets business expectations.

V. Gherkin Syntax in Cucumber

- Gherkin is a domain-specific language used in Cucumber to write feature files.
- It follows a structured syntax that is easy to read and understand by both technical and non-technical stakeholders.
- The Gherkin syntax consists of keywords and plain text, which together form the scenarios and steps in the feature files.

Here are the main keywords used in Gherkin syntax:

1. Feature:

- a. The Feature keyword is used to define a high-level description of the software feature under test.

- b. It typically includes the name of the feature and a brief description.

2. Scenario:

- a. The Scenario keyword is used to define an individual test scenario or test case.
- b. It represents a specific use case or behaviour that needs to be tested.

3. Given, When, Then, And, But:

- a. These keywords are used to define steps within a scenario.
- b. They describe the preconditions (Given), actions (When), and expected outcomes (Then) of the test. Additional steps can be specified using the and But keywords.

4. Background:

- a. The Background keyword is used to define a set of common preconditions that are applicable to all scenarios within a feature.
- b. It helps to avoid repeating the same steps in each scenario.

5. Scenario Outline:

- a. The Scenario Outline keyword is used when you have multiple similar scenarios with different inputs or data sets.
- b. It allows you to create a template scenario with placeholders that can be replaced with actual values from a data table.

- These are some of the key elements of Gherkin syntax used in Cucumber.
- The structured format of Gherkin makes it easy to define and understand test scenarios, promoting collaboration between team members and improving the clarity and readability of the feature files.

VI. Step Definition for Cucumber Feature File

- In Cucumber, step definitions are the actual code implementation of the steps defined in the feature file.
- Step definitions are written in a programming language (such as Java) and are responsible for executing the actions and validations specified in the feature file.

Here is an example of a step definition for a login feature in Cucumber with WebDriver:

```
import cucumber.api.java.en.Given;
import cucumber.api.java.en.When;
import cucumber.api.java.en.Then;
import org.openqa.selenium.By;
import org.openqa.selenium.WebDriver;
import org.openqa.selenium.chrome.ChromeDriver;

public class LoginStepDefinitions {

    private WebDriver driver;

    @Given("^I am on the login page$")
    public void iAmOnLoginPage() {
        // Set up WebDriver and navigate to the login page
        System.setProperty("webdriver.chrome.driver",
            "path/to/chromedriver");
        driver = new ChromeDriver();
        driver.get("https://example.com/login");
    }
}
```

```
@When("^I click on the login button$")
public void iClickOnLoginButton() {
    // Click on the login button
    driver.findElement(By.id("login-button")).click();
}

@Then("^I should be redirected to the dashboard page$")
public void iShouldBeRedirectedToDashboardPage() {
    // Verify that the dashboard page is displayed
    String expectedTitle = "Dashboard";
    String actualTitle = driver.getTitle();
    Assert.assertEquals(expectedTitle, actualTitle);
}

@After
public void tearDown() {
    // Quit the WebDriver instance
    if (driver != null) {
        driver.quit();
    }
}
}
```

- In the above example, each step in the feature file is mapped to a corresponding method in the Step Definitions class.
- The @Given, @When, and @Then annotations are used to specify the matching step text.
- Inside each method, the WebDriver actions and assertions are performed to simulate the behaviour described in the feature file.
- Note that this is just a basic example, and in a real-world scenario, you would have more complex step definitions with additional validations, data handling, and interaction with the application.
- Make sure to include the necessary dependencies for Cucumber and WebDriver in your project, and configure the WebDriver for your chosen browser (e.g., ChromeDriver) before running the tests.
- Remember to also handle any necessary setup and teardown operations, such as initializing and quitting the WebDriver instance, using @Before and @After annotations respectively.
- This is just a simplified example to demonstrate the concept of step definitions in Cucumber.
- In practice, you would have more comprehensive step definitions to cover all the steps in your feature files.

VI. Cucumber Options

- In Cucumber, the @CucumberOptions annotation is used to configure various options for the test execution.
- It allows you to specify features, glue code, tags, output formats, and other settings.

Here's an example of using @CucumberOptions with WebDriver:

```
import cucumber.api.CucumberOptions;
import cucumber.api.junit.Cucumber;
import org.junit.runner.RunWith;

@RunWith(Cucumber.class)
@CucumberOptions(
    features = "src/test/resources/features",
    glue = "com.example.steps",
    tags = "@smoke",
    plugin = {"pretty", "html:target/cucumber-reports"}
)
public class TestRunner {
    // This class should be empty
}
```

In the above example:

- features specifies the path to the directory or file containing the feature files. Here, it is set to

"src/test/resources/features" to indicate that the feature files are located in the "features" directory under the "test/resources" folder.

- glue specifies the package or class where the step definitions are located. In this case, it is set to "com.example.steps".
- tags allows you to run specific scenarios or features based on tags. Here, it is set to "@smoke" to only execute scenarios tagged with "@smoke".
- plugin specifies the output formats for the test execution report. In this example, it is set to generate a pretty console output and an HTML report in the "target/cucumber-reports" directory.
- You can include additional options in the @CucumberOptions annotation as per your requirement. Some common options include monochrome to display console output in a readable format, dryRun to validate the step definitions without executing the actual tests, and strict to fail the execution if any undefined or pending steps are found.
- Make sure to adjust the package names, file paths, and other configurations according to your project structure.
- To execute the Cucumber tests with these options, you can run the TestRunner class as a JUnit test or use a build tool like Maven or Gradle to run the tests from the command line.
- Note: The example provided here uses JUnit as the test runner.

- If you're using a different test runner, the syntax may vary slightly, but the concept of specifying the options using @CucumberOptions remains the same.

VII. Scenario Outline

- In Cucumber, a Scenario Outline is used when you want to run the same scenario multiple times with different sets of data.
- It allows you to create a template scenario with placeholders, and then provide different values for those placeholders using Examples tables.
- This is especially useful when you have test cases that follow a similar structure but require different input data.

Here's an example of using a Scenario Outline in Cucumber with WebDriver:

Scenario Outline: Login Functionality

Given I am on the login page

When I enter "<username>" and "<password>"

And I click the Login button

Then I should be logged in successfully

Examples:

username	password
user1	password1
user2	password2
user3	password3

- In the above example, the Scenario Outline is defined with the title "Login Functionality".
- It consists of several steps like navigating to the login page, entering the username and password, clicking the Login button, and verifying successful login.
- The Examples table below the Scenario Outline defines the different sets of input data for each run of the scenario.
- Each row in the Examples table represents a separate execution of the scenario, with the placeholders "<username>" and "<password>" replaced with the corresponding values.
- When you run this scenario, Cucumber will execute it three times, once for each row in the Examples table.
- This allows you to test the login functionality with different usernames and passwords without duplicating the scenario.
- In the step definitions, you need to provide the implementation for each step using regular expressions or predefined step definitions.
- The placeholders "<username>" and "<password>" in the Scenario Outline will be automatically replaced with the values from the Examples table in each execution.
- Using a Scenario Outline with Examples tables helps in reducing code duplication and allows for more efficient testing of similar scenarios with different data sets.

VIII. Data tables in cucumber

- In Cucumber, data tables are used to provide tabular data as input to a step in a scenario.
- Data tables are useful when you need to pass multiple values or sets of data to a step, such as when testing data-driven scenarios.

Here's an example of using a data table in Cucumber with WebDriver:

Scenario: Add Products to Cart

Given I am on the product listing page

When I add the following products to the cart:

Product Name	Quantity
--------------	----------

iPhone	2
--------	---

MacBook	1
---------	---

Apple Watch	3
-------------	---

Then the products should be added to the cart

- In the above example, the step "When I add the following products to the cart:" expects a data table as input.
- The data table is provided right below the step, and it consists of two columns: "Product Name" and "Quantity".
- Each row in the data table represents a separate product and its corresponding quantity.

- When Cucumber executes this scenario, it will pass the data table to the step definition method as an argument.
- You can then access the values in the data table and perform the necessary actions in your step definition code.

Here's an example of how you can implement the step definition for the above scenario using WebDriver and Java:

```
@When("I add the following products to the cart:")
public void addProductsToCart(DataTable dataTable) {
    List<Map<String, String>> products =
    dataTable.asMaps(String.class, String.class);

    for (Map<String, String> product : products) {
        String productName = product.get("Product Name");
        String quantity = product.get("Quantity");

        // Perform the necessary actions to add the product to the cart
        // Use the values of productName and quantity in your code
    }
}
```

- In the above step definition code, the DataTable is converted to a List of Maps using the asMaps() method.

- Each map in the list represents a row in the data table, where the column headers are used as keys and the cell values are used as values.
- You can then iterate over the list and access the values using the corresponding keys.
- Using data tables in Cucumber allows you to provide structured and tabular input to your scenarios, making it easier to test different sets of data and validate the expected behaviour of your application.

IX. Advantages of Cucumber

- Cucumber WebDriver offers several advantages that make it a popular choice for behavior-driven development (BDD) and test automation:

1. Readability:

- a. Cucumber feature files written in Gherkin syntax are highly readable and understandable by both technical and non-technical stakeholders.
- b. This promotes collaboration and ensures clear communication between team members.

2. Business-Readable Tests:

- a. With Cucumber, tests are written in a business-readable format, allowing stakeholders and domain experts to contribute to the test scenarios.

- b. This promotes a shared understanding of the requirements and helps bridge the gap between technical and non-technical team members.

3. Reusability:

- a. Cucumber promotes the concept of reusable step definitions, allowing you to write and reuse steps across different feature files and scenarios.
- b. This reduces duplication and improves maintainability of the test codebase.

4. Test Documentation:

- a. Cucumber feature files serve as living documentation of the application's behavior.
- b. By reading the feature files, you can gain insights into the expected behaviour of the application and use them as a reference point for discussions, troubleshooting, and requirements validation.

5. Integration with WebDriver:

- a. Cucumber seamlessly integrates with WebDriver, allowing you to automate web-based scenarios and interact with the application's UI.
- b. You can write step definitions that use WebDriver API methods to perform actions on web elements, validate expected behavior, and handle assertions.

6. Test Reporting:

- a. Cucumber generates comprehensive test reports that provide detailed insights into the execution status and test results.
- b. These reports can be easily shared with stakeholders and team members to track the progress of the tests and identify any issues.

7. Test Parallelization:

- a. Cucumber supports parallel execution of scenarios, allowing you to run tests concurrently and reduce the overall test execution time.
- b. This is particularly beneficial when dealing with a large number of scenarios or when executing tests in a distributed environment.

8. Extensibility:

- a. Cucumber is highly extensible, allowing you to integrate it with other tools and frameworks.
 - b. You can easily incorporate additional libraries, plugins, or custom code to enhance the capabilities of your test automation framework.
- Overall, Cucumber WebDriver enables collaboration, improves test readability, and provides a structured approach to behaviour-driven testing.
 - It encourages effective communication among team members, promotes reusability, and enhances the overall quality of the testing process.

X. Implementing BDD Framework using Cucumber

- Implementing a BDD (Behavior-Driven Development) framework using Cucumber WebDriver involves the following steps:

1. Set up the Project:

- a. Create a new project in your preferred IDE (e.g., Eclipse, IntelliJ) and add the necessary dependencies for Cucumber and WebDriver.
- b. You will need the Cucumber-Java and Cucumber-JUnit libraries, as well as the WebDriver library for your chosen browser (e.g., Selenium WebDriver for Chrome).

2. Create Feature Files:

- a. Write your feature files using the Gherkin syntax. Feature files describe the behavior of the application in a human-readable format.
- b. Each feature file contains scenarios with corresponding steps.

3. Define Step Definitions:

- a. Create step definition classes that map the steps defined in the feature files to the corresponding automation code.
- b. Each step is implemented as a method in a step definition class and performs the necessary actions using WebDriver methods.

4. Configure Test Runner:

- a. Set up the Cucumber test runner class that defines the test execution flow.
- b. This class specifies the location of the feature files and step definition classes, and configures other Cucumber options such as generating reports and formatting.

5. Implement Test Cases:

- a. Write the actual test cases by combining the steps defined in the feature files with the automation logic in the step definition classes.
- b. Use WebDriver methods to interact with the application, perform actions, and validate the expected results.

6. Execute Tests:

- a. Run the Cucumber test runner class to execute the BDD tests.
- b. Cucumber will read the feature files, match the steps with the corresponding step definitions, and execute the automation code.
- c. The test results will be displayed in the console or generated in a report format.

7. Analyze Test Results:

- a. Review the test results to identify any failures or issues.
- b. Cucumber provides detailed information about which steps passed or failed, along with any error messages or stack traces.

8. Enhance the Framework:

- a. Customize and enhance the BDD framework as needed.
 - b. You can add additional features, such as test data management, test configuration, logging, or test environment setup/teardown, to make the framework more robust and scalable.
- By following these steps, you can implement a BDD framework using Cucumber WebDriver.
 - This approach promotes collaboration, improves test readability, and provides a structured way to define and execute your tests based on the expected behaviour of the application.

XI. Introduction to Selenium Grid

- Selenium Grid is a tool in the Selenium suite that allows you to run tests in parallel across different browsers, operating systems, and machines.
- It enables you to distribute your test execution on multiple nodes, reducing the overall execution time and improving the efficiency of your test suite.

Here is a brief explanation of Selenium Grid and its key components:

1. Hub:

- a. The Selenium Grid hub acts as a central control point that manages the distribution of test execution across different nodes.
- b. It receives test requests and delegates them to available nodes for execution.

2. Nodes:

- a. Nodes are the machines or devices where the actual test execution takes place.
- b. Each node has a specific configuration, such as the browser version and operating system, and registers itself with the hub.
- c. Nodes can be located on the same machine as the hub or on remote machines connected to the network.

3. Grid Configuration:

- a. The grid configuration defines the capabilities of each node, including the browser type, version, platform, and any additional configuration parameters.
- b. This information is used by the hub to assign tests to appropriate nodes based on their capabilities.

4. Test Execution:

- a. When a test is triggered, the hub receives the request and looks for an available node that matches the desired capabilities.
- b. Once a compatible node is found, the test script is sent to that node for execution.
- c. The node launches the specified browser and performs the test steps as instructed.

5. Parallel Execution:

- a. Selenium Grid enables parallel execution by distributing tests across multiple nodes simultaneously.
 - b. This allows you to run tests in parallel across different browsers, operating systems, or configurations, resulting in faster test execution and increased test coverage.
- By using Selenium Grid, you can efficiently utilize your test infrastructure resources, reduce test execution time, and ensure compatibility across various browser and operating system combinations.
 - It is particularly useful for large-scale test automation projects or when you need to support cross-browser testing.
 - To use Selenium Grid, you need to set up the hub and register the nodes with their respective configurations.
 - Once the hub and nodes are configured, you can execute your tests by specifying the desired capabilities and pointing them to the hub URL.
 - Overall, Selenium Grid is a powerful tool that enhances the scalability and efficiency of your Selenium test automation by allowing parallel execution across different environments.

Here is a high-level overview of the Selenium Grid architecture:

1. Test clients send test requests to the hub, specifying the desired capabilities for test execution.

2. The hub receives the test requests and searches for available nodes that match the requested capabilities.
 3. Once a compatible node is found, the hub assigns the test to that node and provides the necessary instructions.
 4. The node receives the test instructions from the hub and launches the specified browser with the desired capabilities.
 5. The test is executed on the node, and the test results are sent back to the hub.
 6. The hub collects the test results from the nodes and provides them to the test client.
- By leveraging Selenium Grid's distributed architecture, you can execute tests in parallel across multiple nodes, thereby reducing the overall execution time and improving the scalability of your test suite.
 - This allows you to perform cross-browser testing, test on different operating systems, and handle larger test volumes more efficiently.
 - Note that Selenium Grid requires the installation and configuration of both the hub and the nodes to set up the distributed test execution environment.

XII. Deploy a Grid

To deploy a Selenium Grid, you need to set up a hub and one or more nodes. Here are the steps to deploy a Selenium Grid:

1. Set up the Hub:

- Download the Selenium Server standalone JAR file from the Selenium website (<https://www.selenium.dev/downloads/>).
- Open a command prompt or terminal and navigate to the directory where the JAR file is located.
- Start the hub by running the following command:

```
java -jar selenium-server-standalone.jar -role hub
```

2. Set up the Nodes:

- On each machine where you want to run the nodes, ensure that you have the necessary browser and WebDriver configurations in place.
- Open a command prompt or terminal on each machine and navigate to the directory where the Selenium Server standalone JAR file is located.
- Start the node by running the following command:

```
java -jar selenium-server-standalone.jar -role node -hub http://<hub-ip>:<hub-port>/grid/register
```

- Replace <hub-ip> and <hub-port> with the IP address and port of the hub.

3. Verify the Grid Setup:

- Open a web browser and navigate to the hub's console at `http://<hub-ip>:<hub-port>/grid/console`. You should see the hub and connected nodes listed.

4. Configure and Run Tests:

- Update your test scripts or test automation framework to use the remote WebDriver and specify the desired capabilities for the desired browser and platform.
- Use the hub's URL (`http://<hub-ip>:<hub-port>/wd/hub`) as the WebDriver's remote URL.
- With the Selenium Grid deployed, you can now run your tests by connecting to the hub and leveraging the available nodes for parallel and distributed test execution.
- Note: Make sure that the machines running the hub and nodes have network connectivity and can communicate with each other.
- Also, ensure that the necessary firewall settings are configured to allow communication between the hub and nodes.

Interview Questions :

1. What is WebDriver?

A: WebDriver is a web automation framework that allows you to automate browser actions and interact with web elements programmatically.

2. What are the advantages of using WebDriver over Selenium RC?

A: WebDriver offers better performance, supports more browsers and platforms, provides a more natural and intuitive API, and has better handling of dynamic web elements.

3. How do you handle synchronization issues in WebDriver?

A: WebDriver provides explicit waits and implicit waits to handle synchronization issues. Explicit waits allow you to wait for a specific condition to be met, while implicit waits set a default waiting time for all elements.

4. What is the difference between `findElement()` and `findElements()` in WebDriver?

A: The `findElement()` method returns the first matching element found on the page, while the `findElements()` method returns a list of all matching elements. If no elements are found, `findElement()` throws an exception, while `findElements()` returns an empty list.

5. How do you handle dropdowns in WebDriver?

A: To handle dropdowns, you can use the `Select` class in `WebDriver`. It provides methods like `selectByVisibleText()`, `selectByValue()`, and `selectByIndex()` to select options from a dropdown.

6. How do you handle multiple windows or tabs in `WebDriver`?

A: `WebDriver` provides the `switchTo()` method to switch between windows or tabs. You can use methods like `getWindowHandles()`, `getWindowHandle()`, and `switchTo().window()` to handle multiple windows.

7. How do you handle alerts and pop-ups in `WebDriver`?

A: `WebDriver` provides methods like `switchTo().alert()` to switch to an alert and perform actions like accepting or dismissing it. For pop-ups, you can use the `getWindowHandles()` method to handle them.

8. What is the Page Object Model (POM) in `WebDriver`?

A: POM is a design pattern that suggests creating separate classes for web pages in your application. Each class represents a page, and it contains all the necessary elements and methods to interact with that page.

9. How do you handle file uploads in `WebDriver`?

A: WebDriver does not have direct support for file uploads. However, you can use third-party libraries like AutoIt or Robot class to handle file uploads.

10. What are the different types of waits available in WebDriver?

A: WebDriver provides three types of waits: explicit, implicit, and fluent. Explicit waits allow you to wait for a specific condition to be met. Implicit waits set a default waiting time for all elements. Fluent waits provide more flexibility in defining custom conditions.

Assignment

1. Write a WebDriver script to navigate to a website and click on a specific link.
2. Implement a Page Object Model for a login page using WebDriver.
3. Write a WebDriver script to handle a dropdown and select an option based on specific criteria.
4. Implement a test case using WebDriver to validate the functionality of a registration form.
5. Write a WebDriver script to capture a screenshot of a webpage and save it to a specific location.